

Cortex-M Architecture

Computer Engineering 1

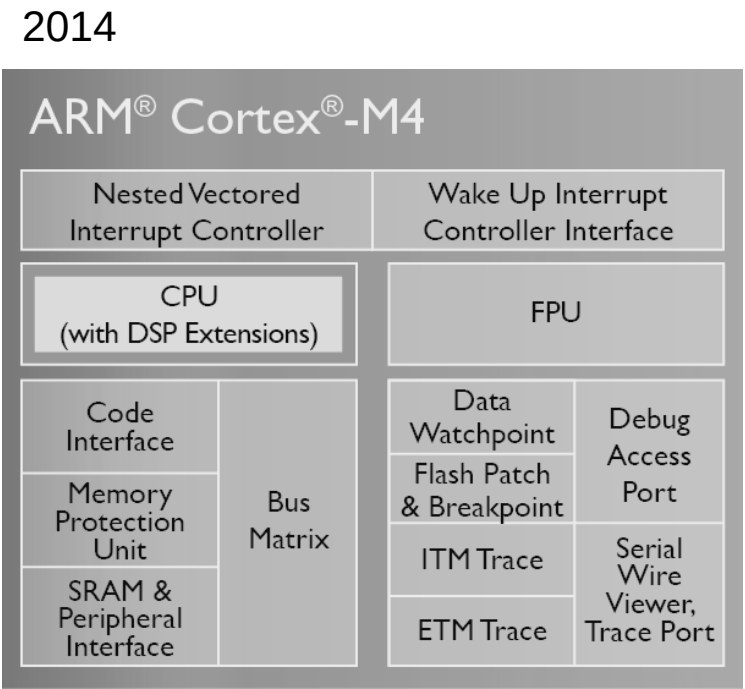
Motivation

– ARM (Cortex-M)



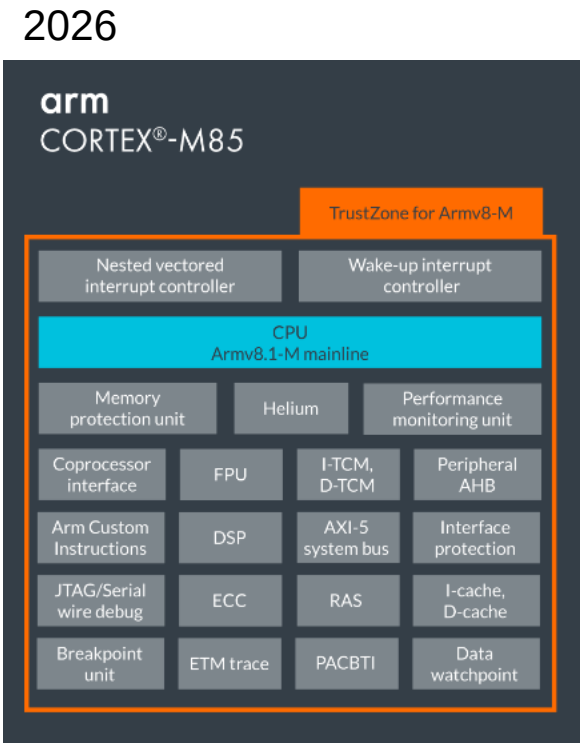
ARM1
25k Transistors
5 MHz

1'400 x
transistors
→
36 x
clock speed



ARMv7 - Cortex-M4
35 Mio Transistors
180 MHz
4 mm²

?? x
transistors
→
4.4 x
clock speed



ARMv8.1-M - Cortex-M85
??? Mio Transistors
800 MHz

Agenda

- **Hardware Platform**
 - STM32F4 and evaluation board, ARM Processor Portfolio
- **CPU Model**
 - Register, ALU, Flags, Control Unit
- **Instruction Set**
 - Assembly, Instruction Types, Cortex-M0
- **Program Execution**
 - Fetch and Execute
- **Memory Map**
 - ARM, ST, CT-Board
- **Integer Types**
 - Sizes, Little Endian vs. Big Endian, Alignment
- **Object File Sections**
 - Code, Data, Stack

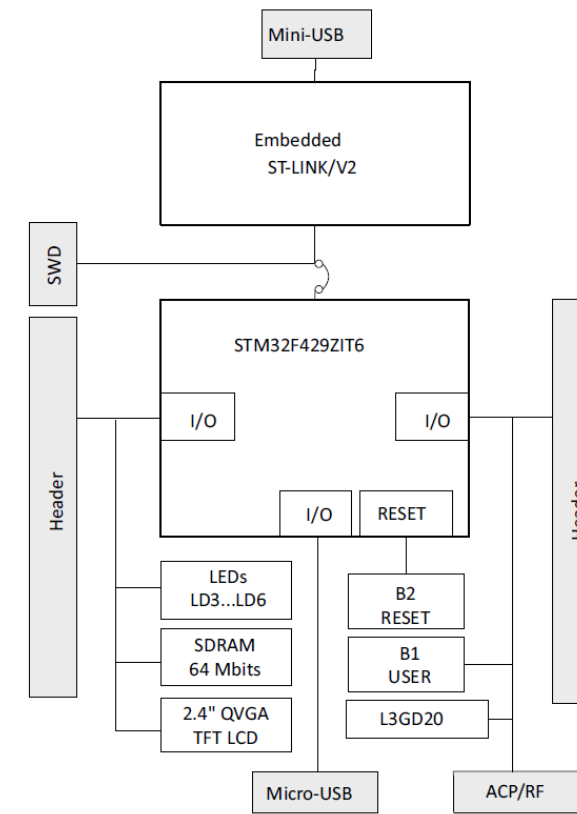
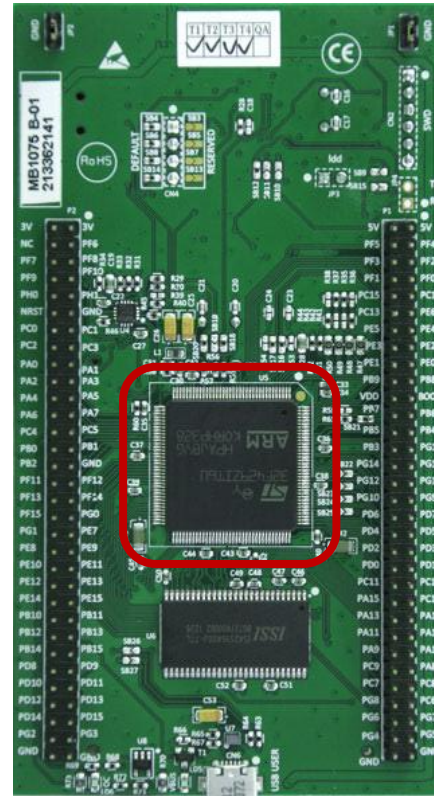
Learning Objectives

At the end of this lesson, you will be able

- to describe what an 'Instruction Set Architecture' is
- to outline the Cortex-M architecture and enumerate the main components and their functions
- to enumerate the instruction type categories
- to understand the structure of the Cortex-M instruction set
- to explain how a processor executes a program
- to recall the registers of the Cortex-M, their layout and their functions
- to interpret, explain and draw a memory map
- to calculate the size in bytes for a memory block given by its start and end address
- to determine the end address of a memory block given by the start address and the number of bytes
- to understand that sizes of integer types in C depend on the architecture and that portability can be enhanced by using the C99 types in `stdint.h`
- to explain the difference between 'little endian' and 'big endian' and to show how multi-byte integer values are mapped to individual bytes
- to list the three typical memory sections of an object file and to explain their content

Hardware Platform

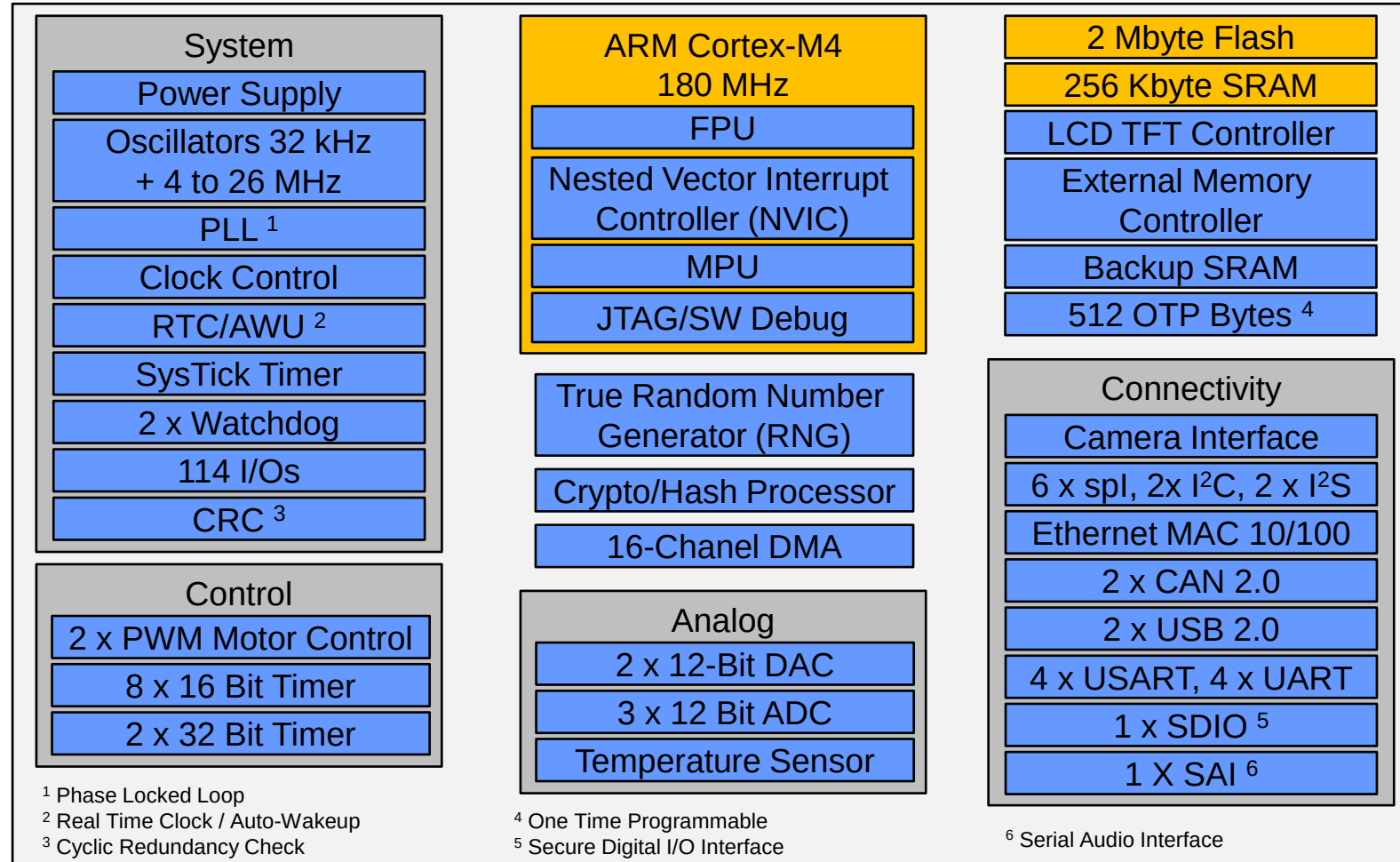
Evaluation board STM32F4-DISCO



source: STMicroelectronics

Hardware Platform

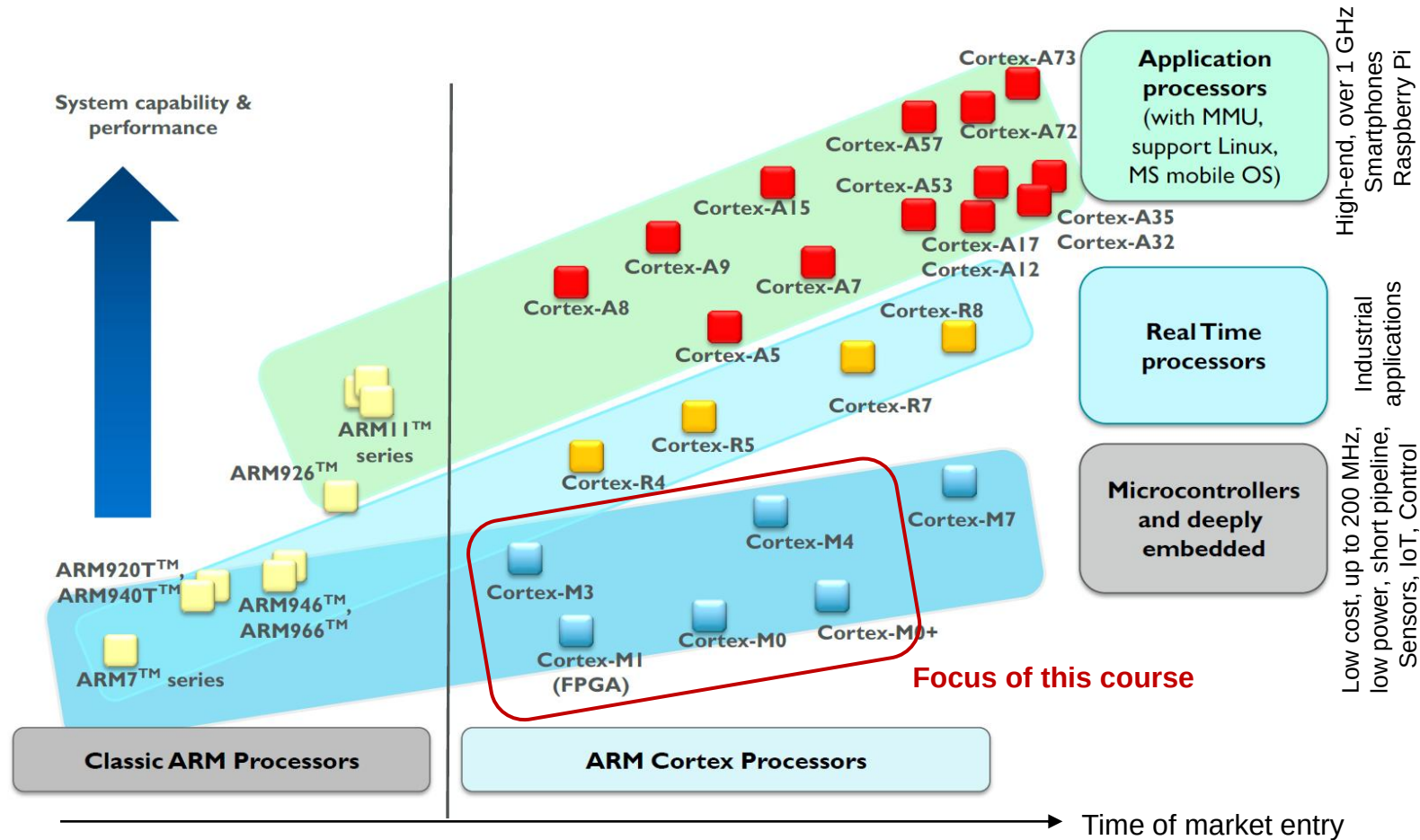
STM32F429I



based on source of STMicroelectronics

Hardware Platform

ARM Processor Portfolio

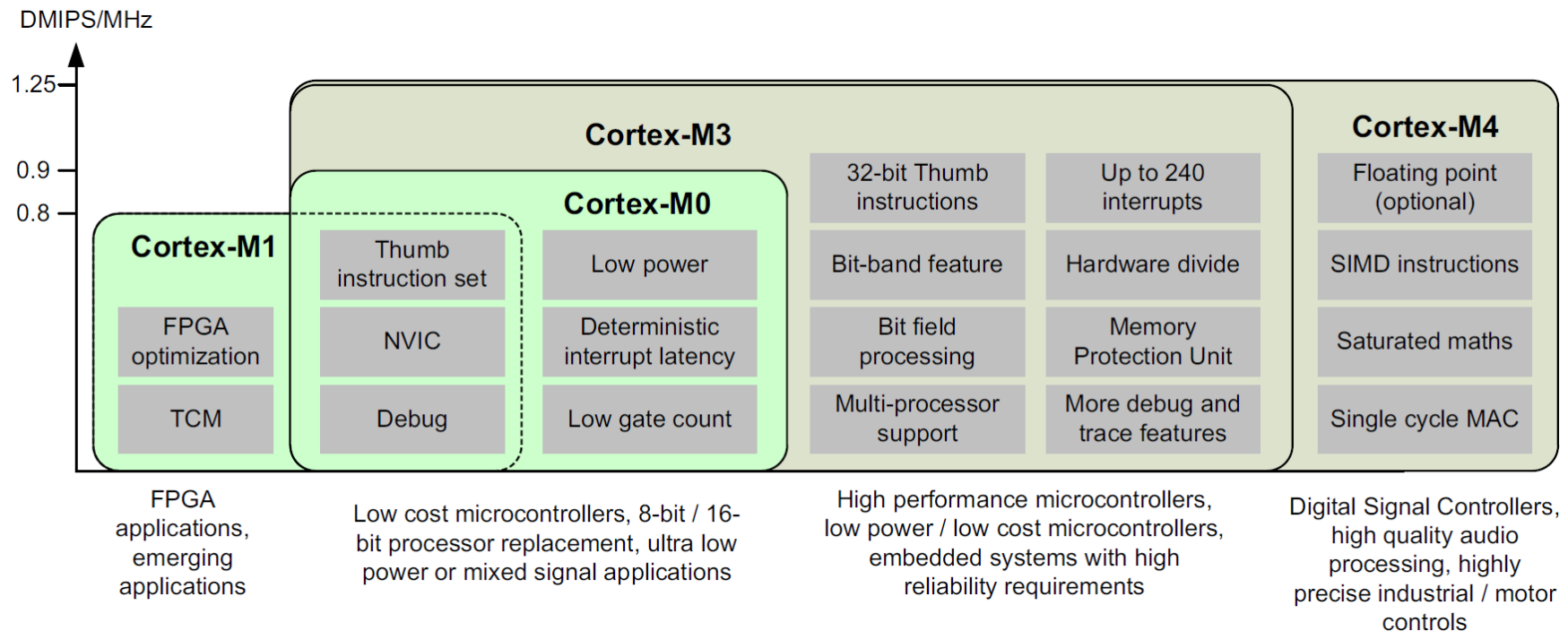


Source: ARM Limited. White paper: ARM® Cortex®-M for Beginners

Hardware Platform

Cortex-M Processor Family

- Hardware used in this course is a Cortex-M4
- But most of the time we only use the simpler Cortex-M0 subset

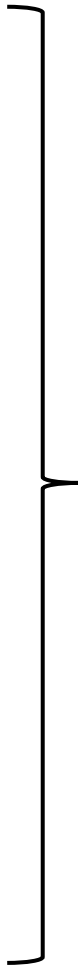


source: Joseph Yiu, The definitive Guide to the Cortex-M0

CPU Model

Instruction Set Architecture (ISA)

- Instruction Set
 - Available instructions?
- Processing width
 - 8-bit/16-bit/32-bit?
- Register set
 - How many registers? Which size?
- Addressing modes
 - How can memory and IO be accessed?
- ARM Cortex-M
 - ARMv6-M → Cortex-M0
 - ARMv7-M → Cortex-M3/M4 (Superset of ARMv6-M)

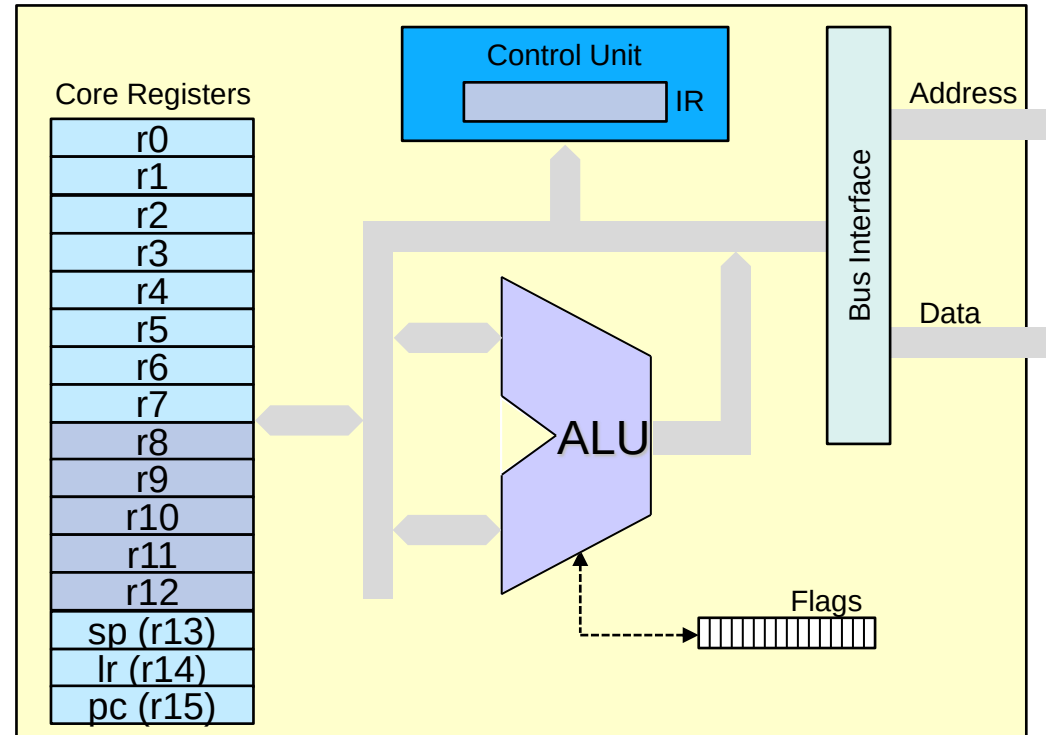


What the programmer
sees of a computer

CPU Model

CPU Components

- Core Registers
- 32-bit ALU
- Flags (APSR)
- Control Unit with IR (Instruction Register)
- Bus Interface

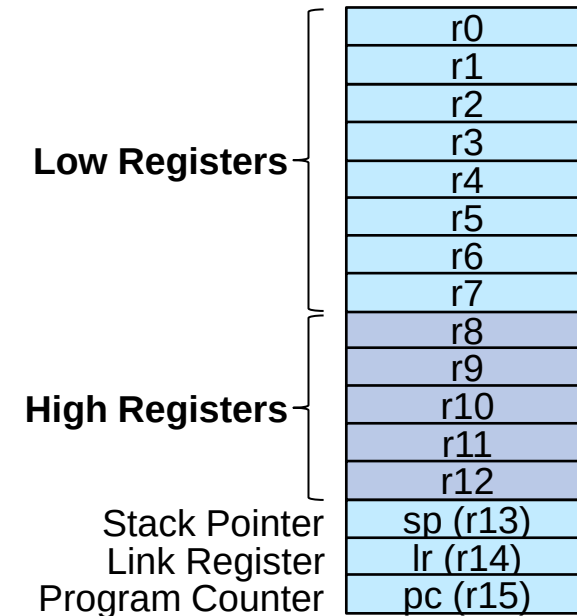


simple CPU model based on the Programmers' Model of the ARM Cortex-Mx CPUs ¹⁾

CPU Model

Core Registers

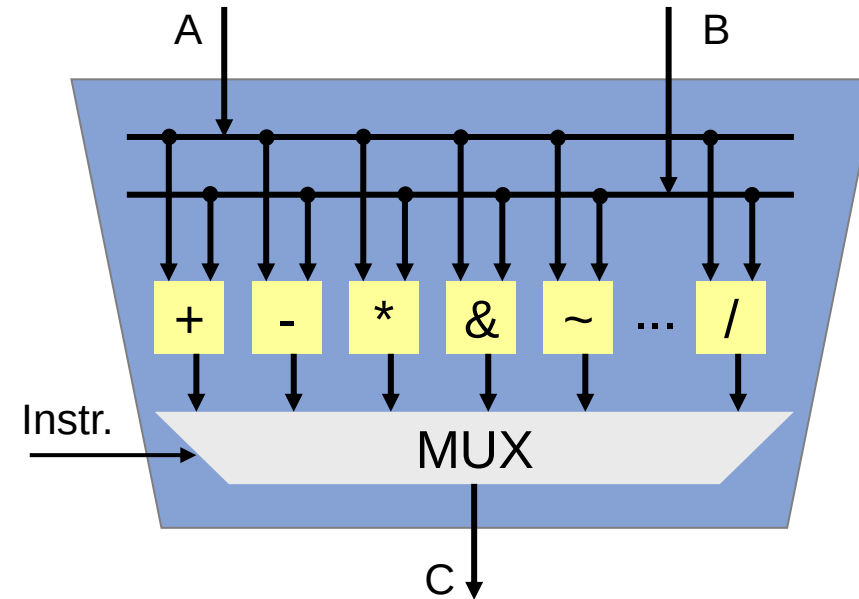
- 32-bit wide
- 13 General-Purpose Registers
 - Low Registers r0 – r7
 - High Registers r8 – r12
 - Used for temporary storage of data and addresses
- Program Counter (r15)
 - Address of next instruction
- Stack Pointer (r13)
 - Last-In First-Out for temporary data storage
- Link Register (r14)
 - Return from procedures



CPU Model

ALU – Arithmetic Logic Unit

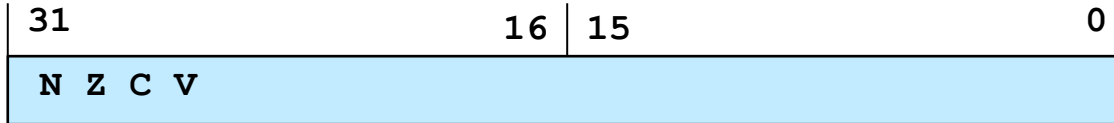
- 32-bit wide data processing unit
 - inputs A and B
 - result C
- integer arithmetic
 - addition / subtraction
 - multiplication / division
 - sign extension
- logic operations
 - AND, NOT, OR, XOR
- shift/rotate
 - left / right



CPU Model

APSr1¹⁾ or Flag-Register

- Bits set by CPU based on results in ALU



N Negative

Z Zero

C Carry

V Overflow

As seen in the IDE

Registers	
Register	Value
Core	
R0	0x20000078
R1	0x20000278
R2	0x20000278
R3	0x20000278
R4	0x08000860
R5	0x00000000
R6	0x00000000
R7	0x00000000
R8	0x00000000
R9	0x00000000
R10	0x08000860
R11	0x08000860
R12	0x00000000
R13 (SP)	0x20000678
R14 (LR)	0x08000243
R15 (PC)	0x08000270
xPSR	0x41000000
N	0
Z	1
C	0
V	0

¹⁾ APSR: Application Processor Status Register

CPU Model

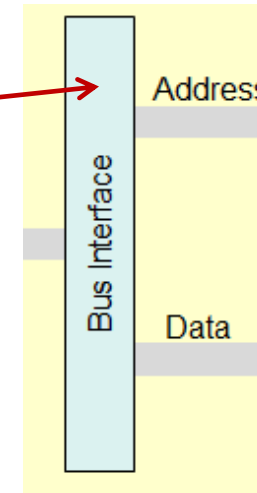
Control Unit / Bus Interface

- **Control Unit**

- Instruction Register (IR)
 - Machine code (opcode) of instruction that is currently being executed
- Controls execution flow based on instruction in IR
- Generates control signals for all other CPU components

- **Bus Interface**

- Interface between internal CPU bus and external system-bus
 - contains registers to store addresses



Instruction Set

- Processors interpret binary coded instructions
 - But binary is hard for programming
 - Easier: instructions in human readable text form
 - → assembly
 - Assembler (tool) does the translation
 - assembly → binary

`adds r0,r0,r1`



0001'1000'0100'0000
=
0x1840

Instruction Set

Assembly Program

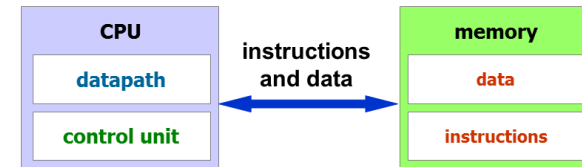
- Label (optional)
- Instruction (Mnemonic)
- Operands
- Comment (optional)

Label	Instr.	Operands	Comments
demoprg:			
	movs	r0,#0xA5	@ copy 0xA5 into register r0
	movs	r1,#0x11	@ copy 0x11 into register r1
	adds	r0,r0,r1	@ add contents of r0 and r1
			@ store result in r0
	ldr	r2,#0x2000	@ load 0x2000 into r2
	str	r0,[r2]	@ store content of r0 at @ the address given by r2

Instruction Set

Instruction Types

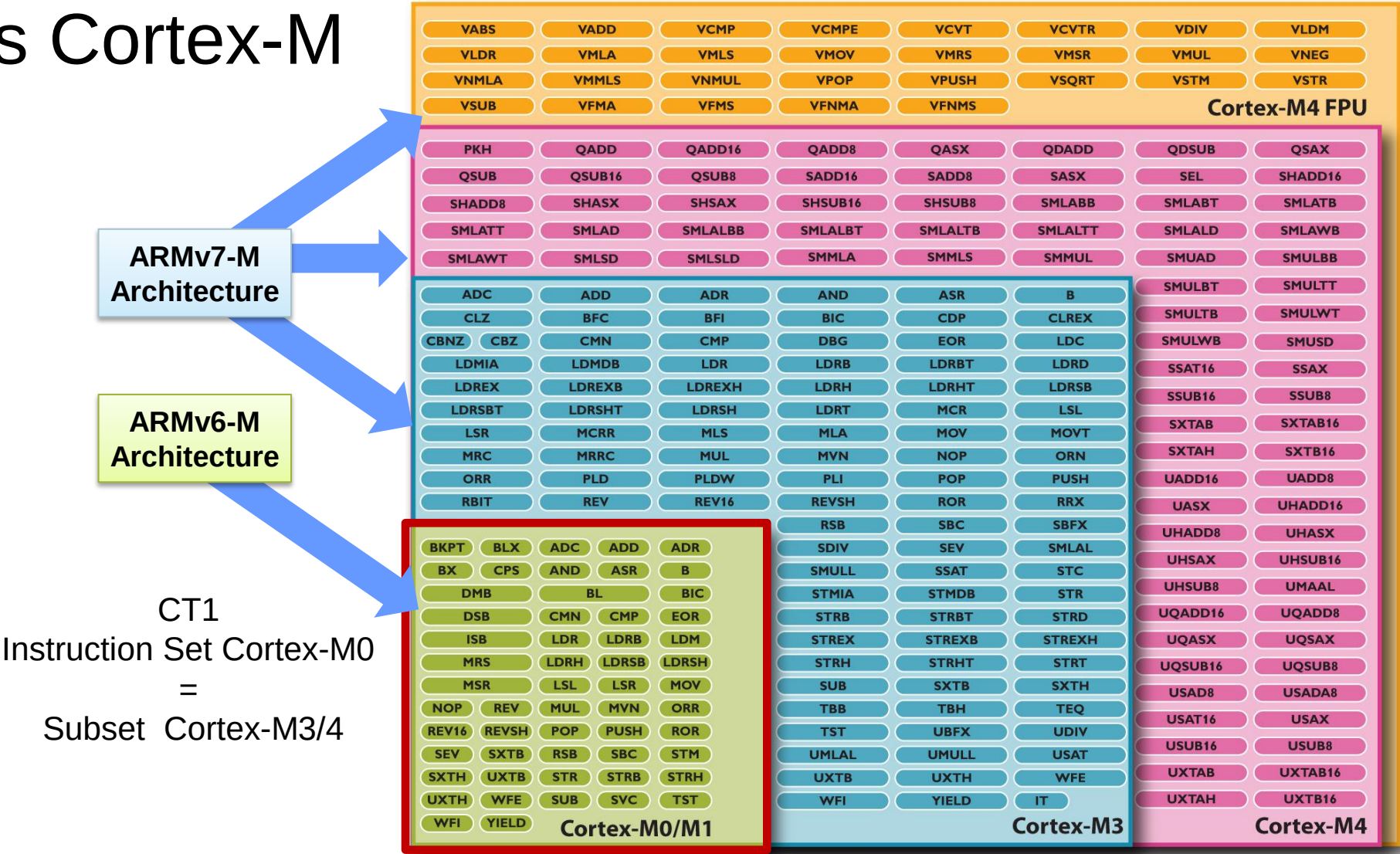
- Data transfer
 - Copy content of one register to another register
 - Load registers with data from memory
 - Store register contents into memory
- Data processing
 - Arithmetic operations → + - * / ...
 - Logic operations → AND, OR, ...
 - Shift / rotate operations
- Control flow
 - Branches
 - Function calls
- Miscellaneous (various)



Type	Frequency	
Data transfer	43%	
Control flow	23%	
Arithmetic	15%	} Data processing
Compare	13%	
Logical	5%	
Miscellaneous	1%	

Instruction Set

Instructions Cortex-M



source: ARM

Instruction Set

Overview Cortex-M0

Instruction Type	Instructions
Move	mov, movs
Load/Store	ldr, ldrb, ldrh, ldrsb, ldrsh, ldm, str, strb, strh, stm
add, Subtract, Multiply	add, adds, adcs, adr, sub, subs, sbcs, rsbs, muls
Compare	cmp, cmn
Logical	ands, eors, orrs, bics, mvns, tst
Shift and Rotate	lsls, lsrs, asrs, rors
Extend	sxth, sxtb, uxth, uxtb
Reverse	rev, rev16, revsh
Branch	b, bl, b{cond}, bx, blx
Stack	pop, push
Processor State	bkpt, cps, mrs, msr, svc
No Operation	nop
Hint / Synchronization	dmb, dsb, isb, sev, wfe, wfi, yield

Instruction Set

Cortex-M0: 16-bit Thumb instruction encoding

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0		
0	0	0	opcode			imm5				Rm			Rd		shift by immediate, move register		
0	0	0	1	1	0	opc	Rm			Rn		Rd		add/subtract register			
0	0	0	1	1	1	opc	imm3			Rn		Rd		add/subtract immediate			
0	0	1	opcode		Rdn			imm8								add/sub./comp./move immediate	
0	1	0	0	0	0	opcode			Rm			Rd		data-processing register			
0	1	0	0	0	1	opcode		DN	Rm			Rd		special data processing			
0	1	0	0	0	1	1	1	L	Rm			0	0	0	branch/exchange		
0	1	0	0	1	Rt			pc-relative imm8								load from literal pool	
0	1	0	1	opcode		Rm			Rn		Rt		load/store register offset				
0	1	1	B	L	imm5			Rn		Rt		load/store word/byte imm. offset					
1	0	0	0	L	imm5			Rn		Rt		load/store halfword imm. offset					
1	0	0	1	L	Rt			sp-relative imm8								load from or store to stack	
1	0	1	0	sp	Rd			imm8								add to sp or pc	
1	0	1	1	x	x	x	x	x	x	x	x	x	x	x	x	miscellaneous	
1	1	0	0	L	Rn			register list								load/store multiple	
1	1	0	1	cond			imm8								conditional branch		
1	1	0	1	1	1	1	0	x	x	x	x	x	x	x	x	undefined instruction	
1	1	0	1	1	1	1	1	imm8								service (system) call	
1	1	1	0	0	imm11											unconditional branch	
1	1	1	0	1	x	x	x	x	x	x	x	x	x	x	x	32-bit instruction	
1	1	1	1	x	x	x	x	x	x	x	x	x	x	x	x	32-bit instruction	

Note: There are some inconsistencies in ARM's use of the notation *Rd* (destination) and *Rt* (target). The slides use the ARMv6-M Architecture Reference Manual (ARM DDI 0419C (ID092410)) as a reference.

Instruction Set

Cortex-M0 Example Program

- Assembler converts each Assembly instruction to 16-bit opcode
- Memory addresses in steps of 2 (Zweierschritte)
 - Reason: opcodes are 16-bit (two bytes) long, e.g. 0x20A5

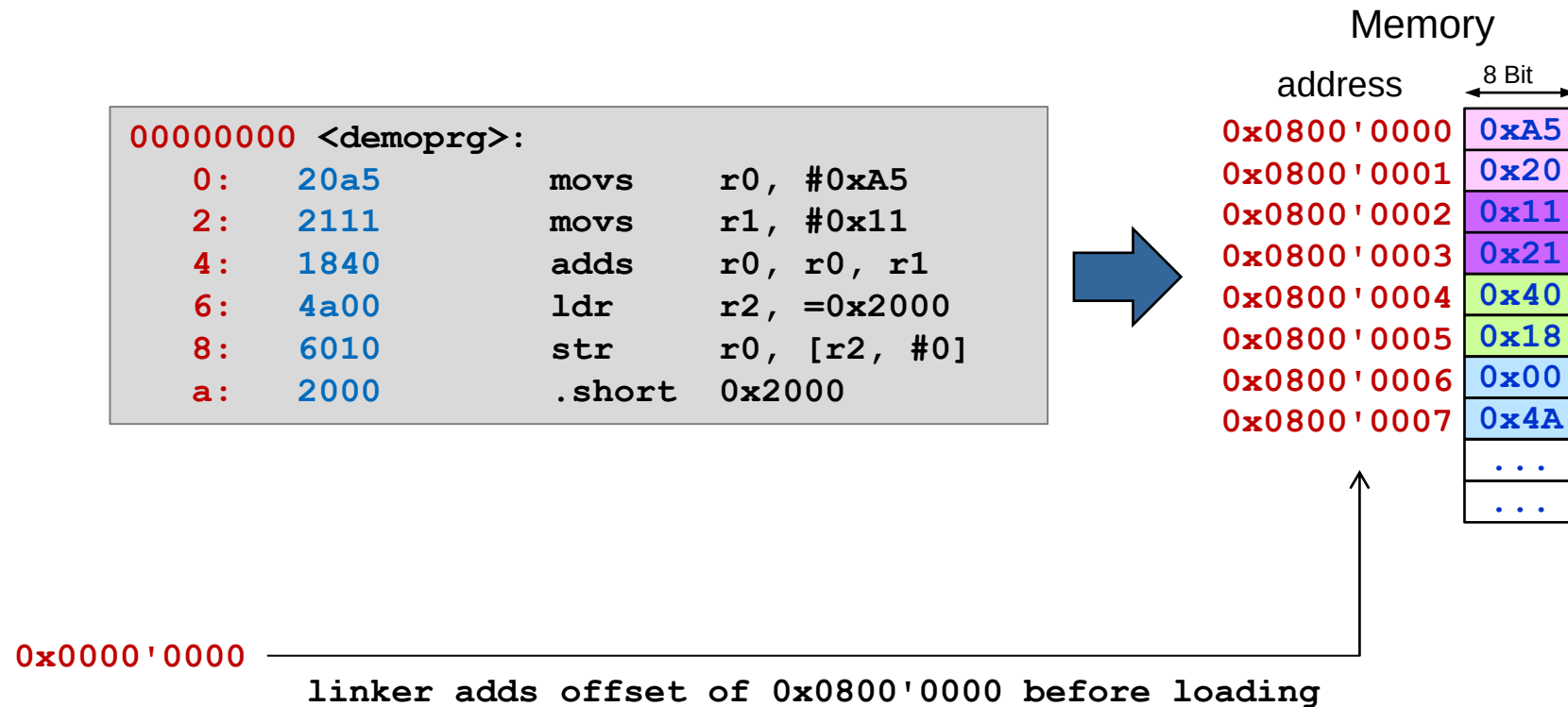
Memory address	Opcode			Listfile
00000000	<demoprg>:			
0:	20a5	movs	r0, #0xA5	@ copy 0xA5 into r0
2:	2111	movs	r1, #0x11	@ copy 0x11 into r1
4:	1840	adds	r0, r0, r1	@ add contents of r0 and r1
				@ store result in r0
6:	4a00	ldr	r2, =0x2000	@ load address into r2 1)
8:	6010	str	r0, [r2, #0]	@ store content of r0 at
				@ the address given by r2
a:	2000	.short	0x2000	

Binary ← generated by Assembler (tool) **Source code**

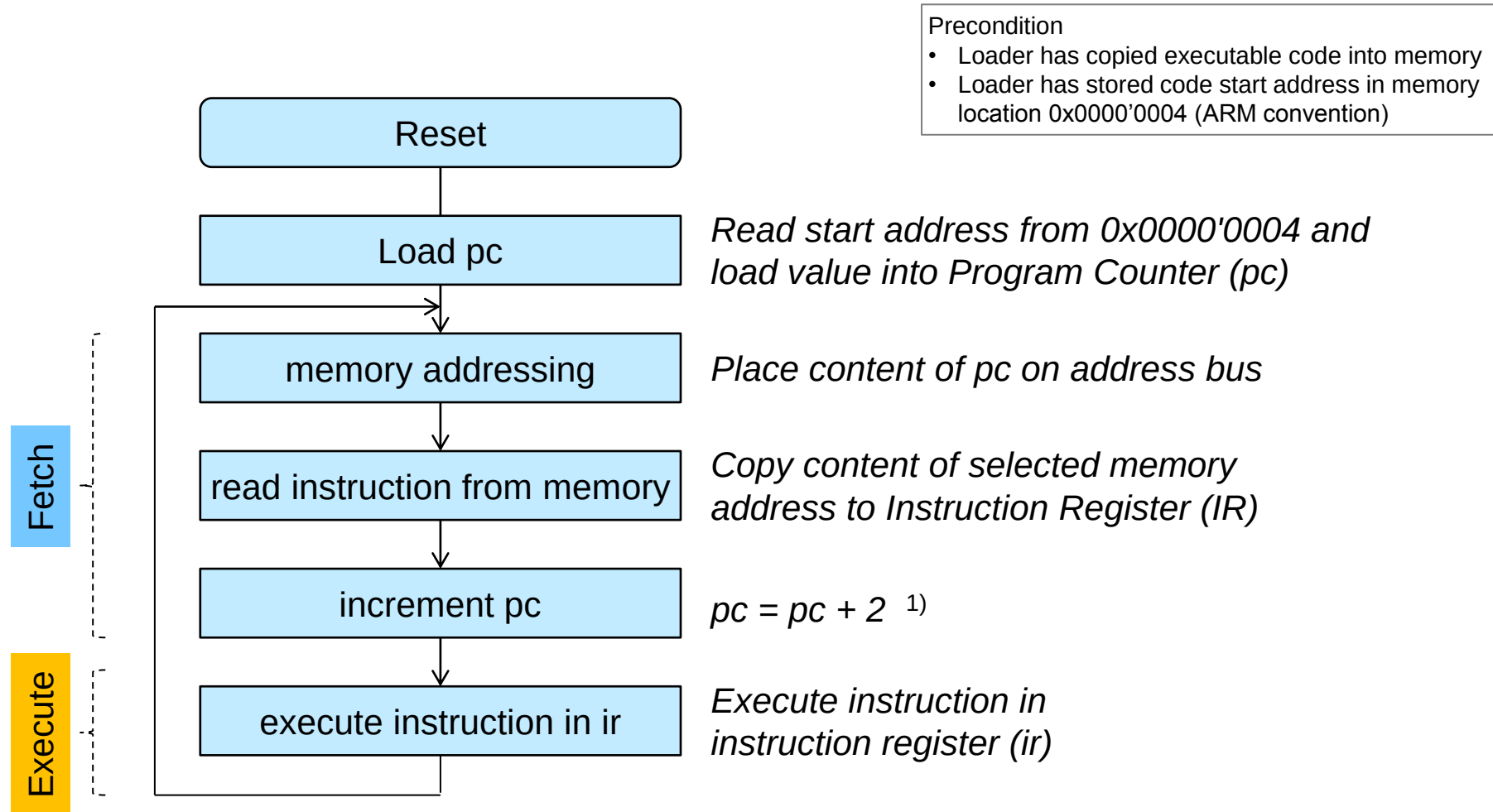
Instruction Set

Load program into memory

- Example assumes code area in memory at address 0x0800'0000 ¹⁾

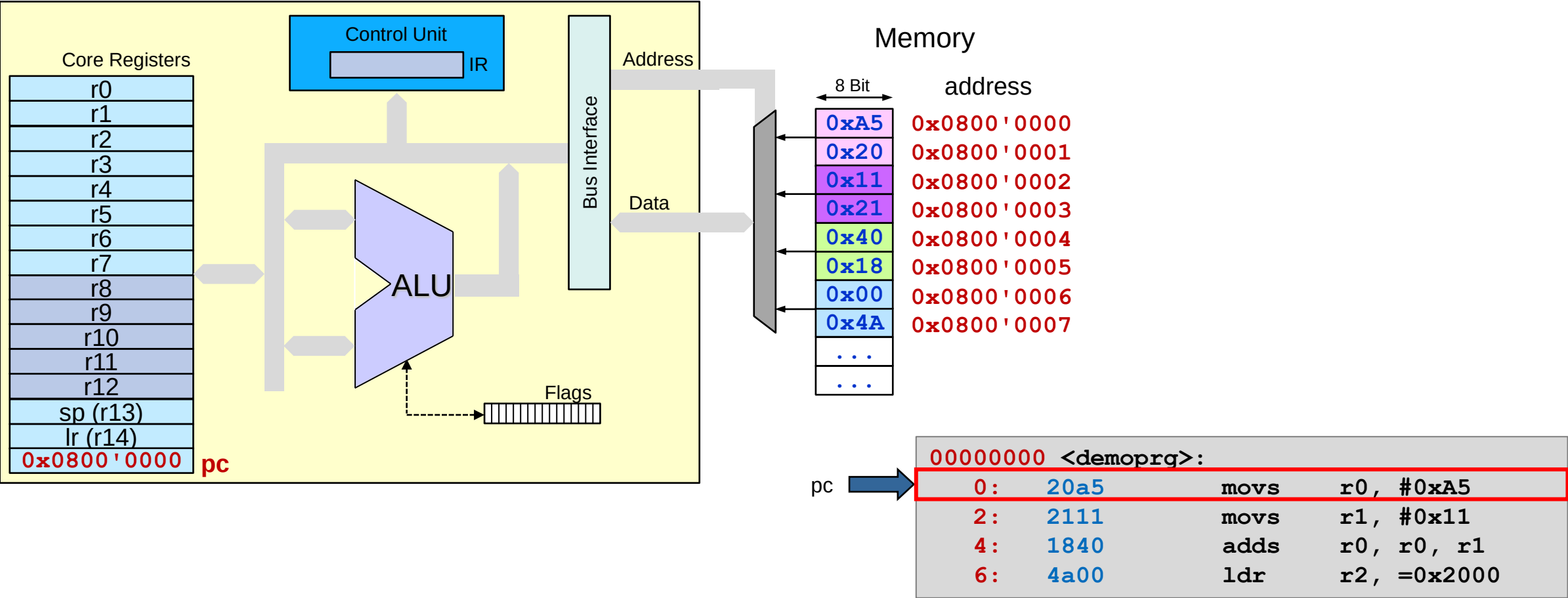


Program Execution Sequence



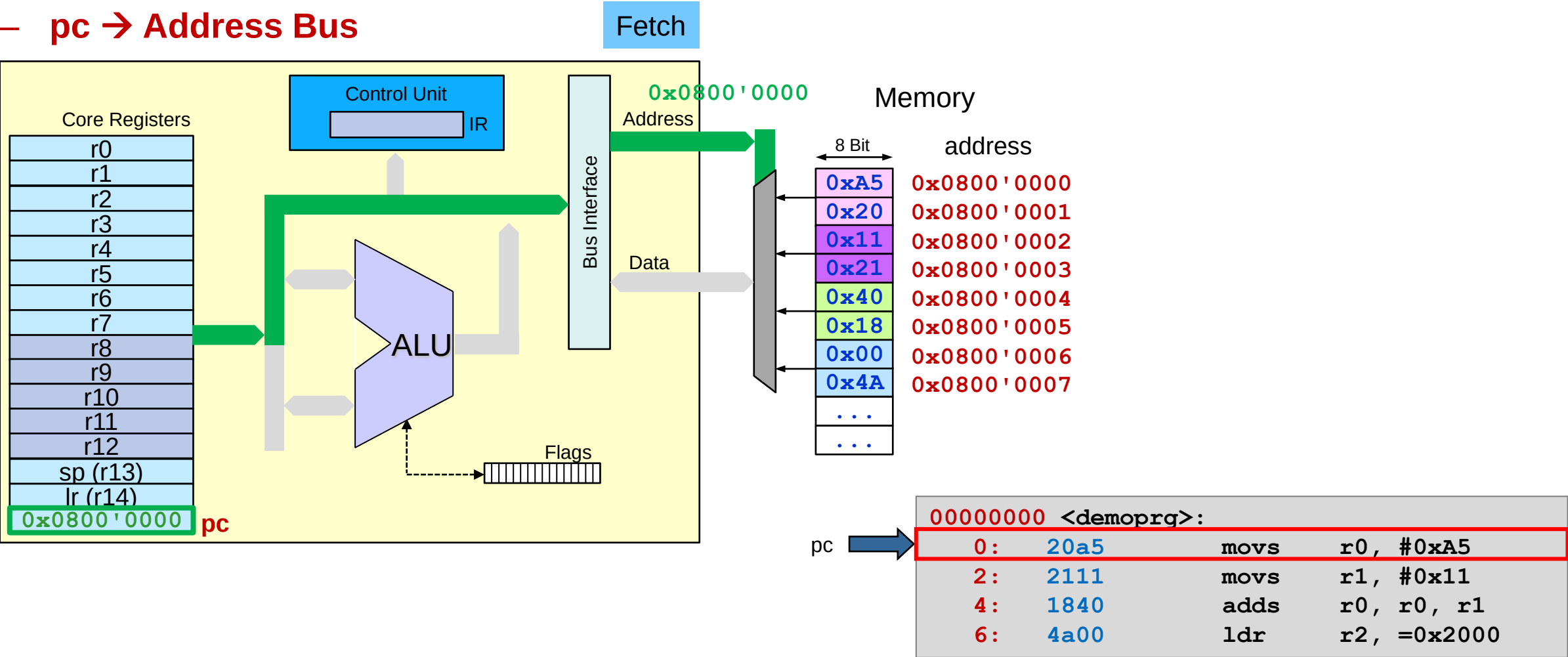
Program Execution

– "Reset": Read 0x0000'0004 → pc



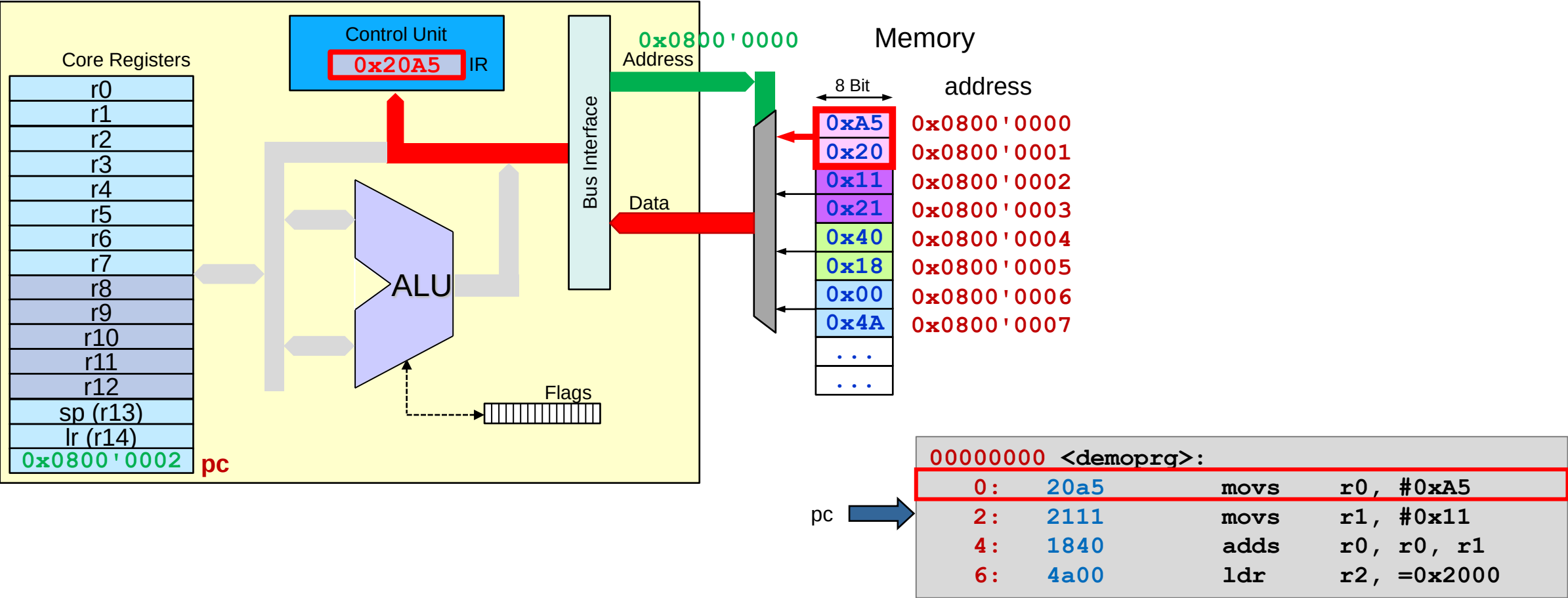
Program Execution

– pc → Address Bus



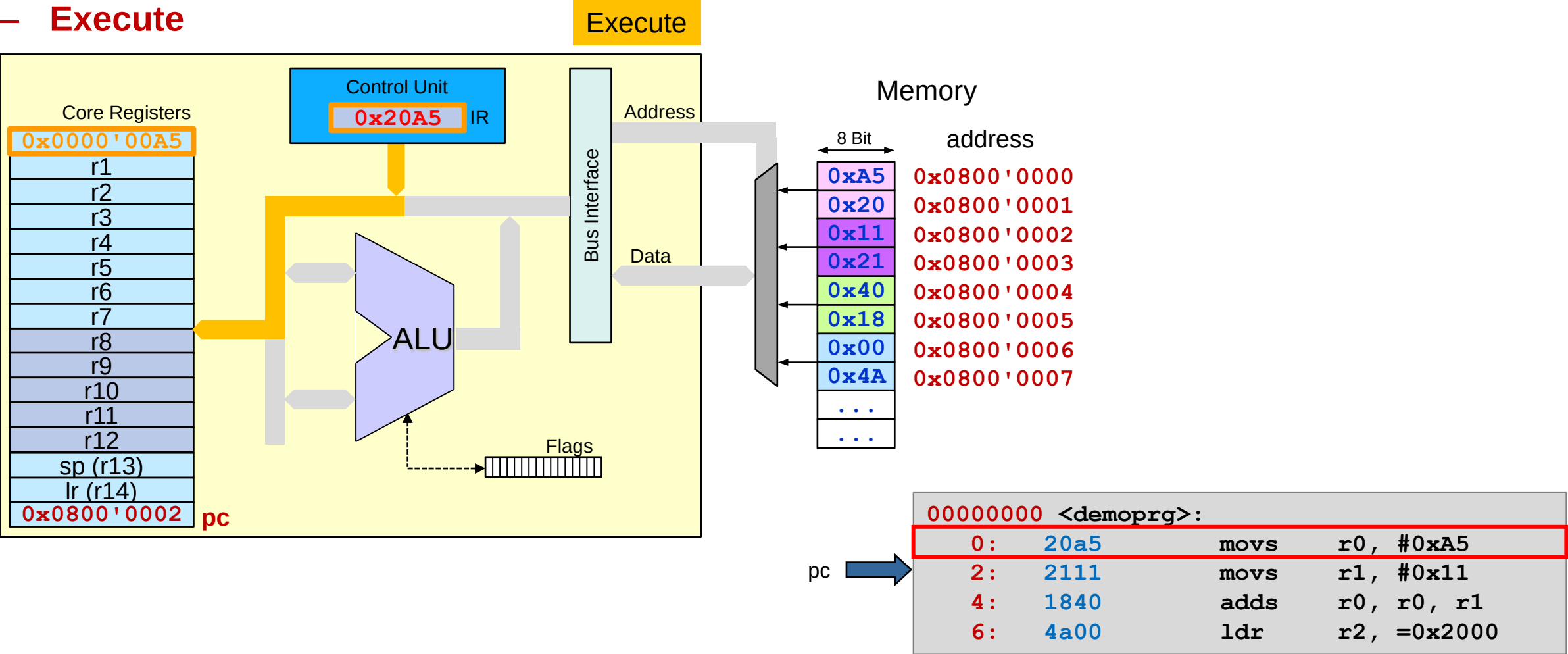
Program Execution

– Read Instruction / Increment pc Fetch



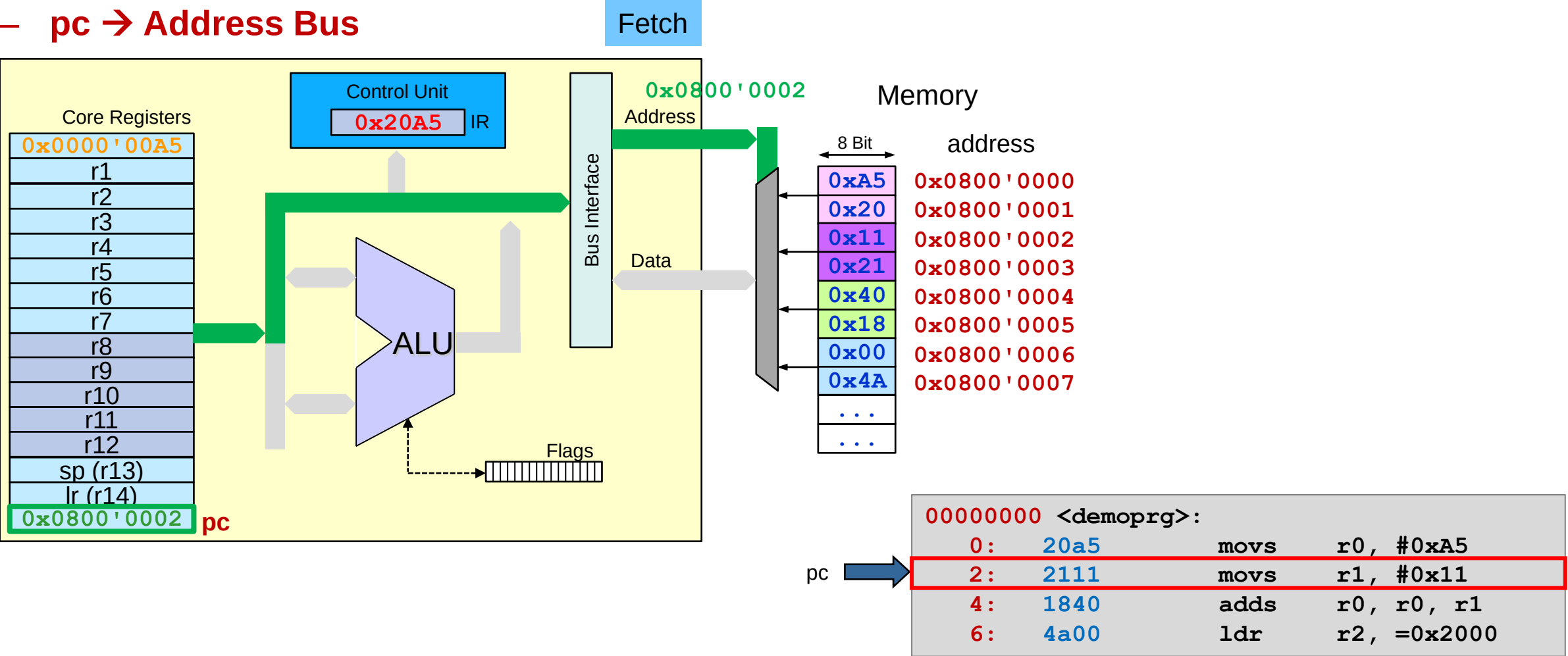
Program Execution

– Execute



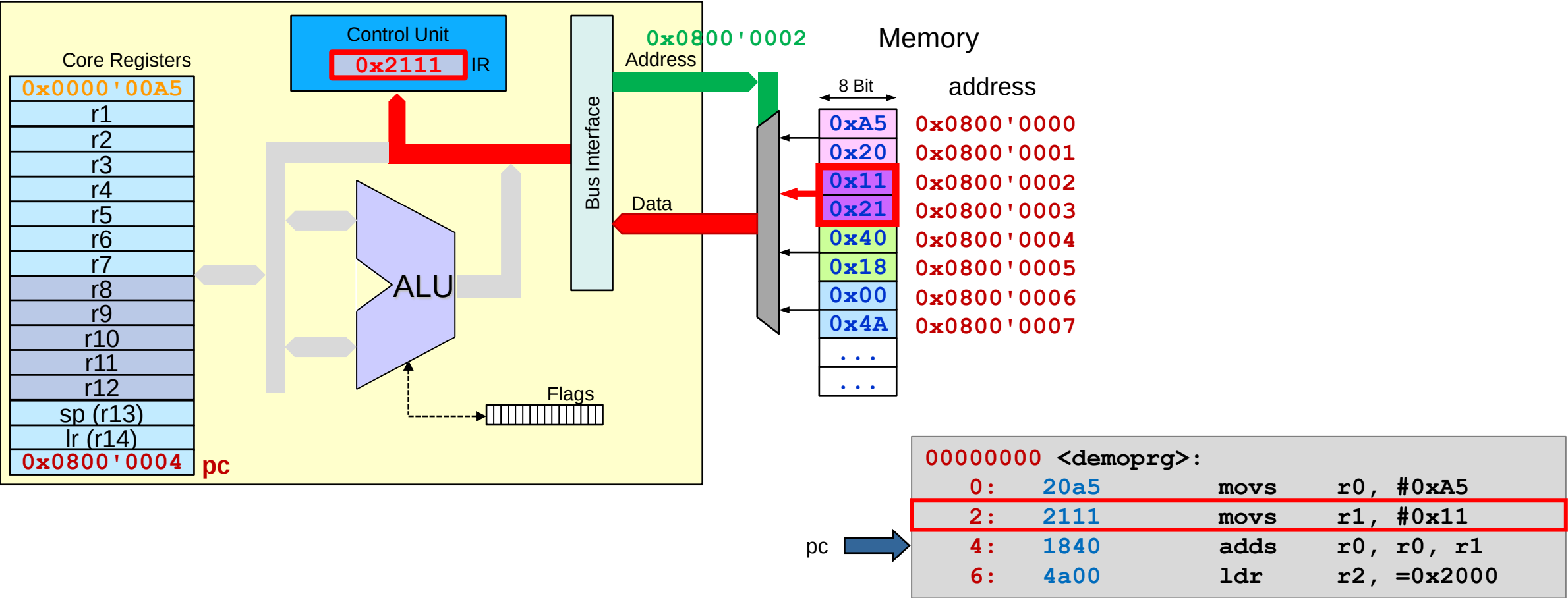
Program Execution

– pc → Address Bus



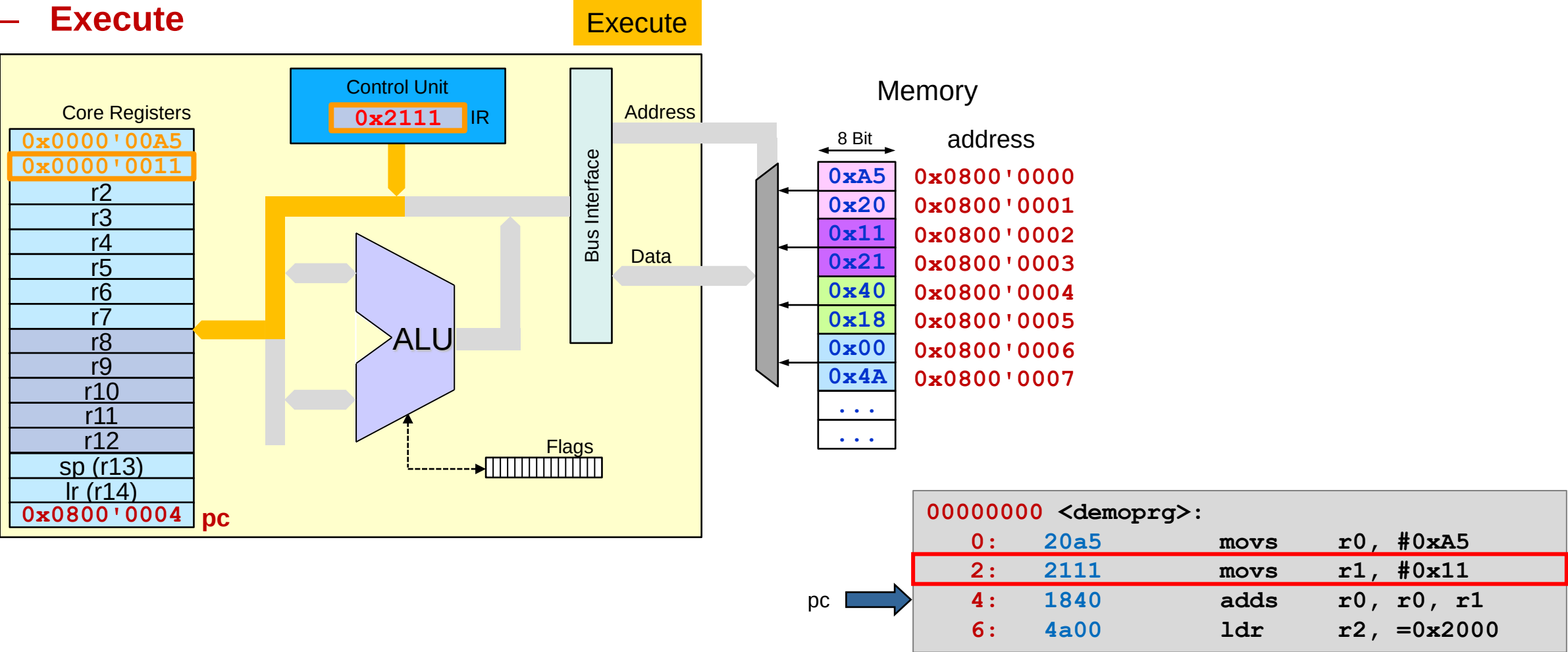
Program Execution

– Read Instruction / Increment pc Fetch



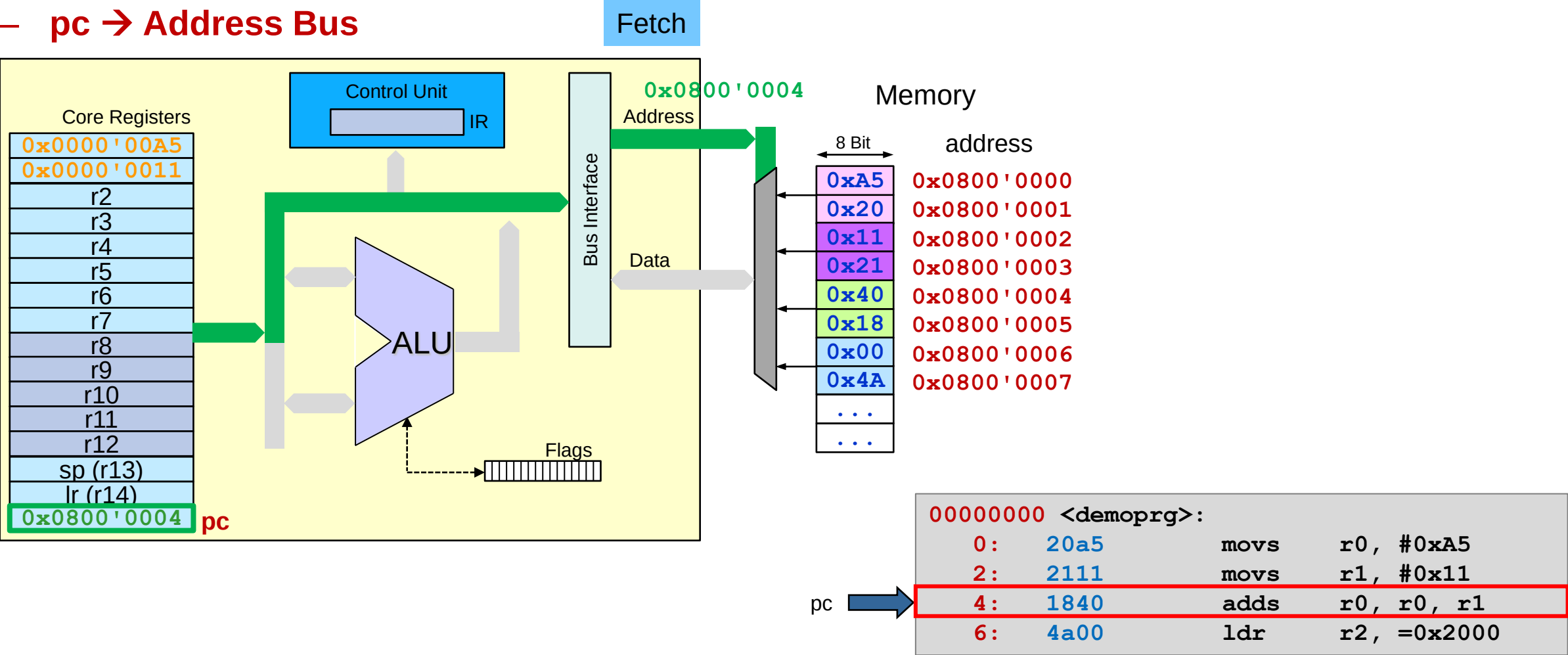
Program Execution

– Execute



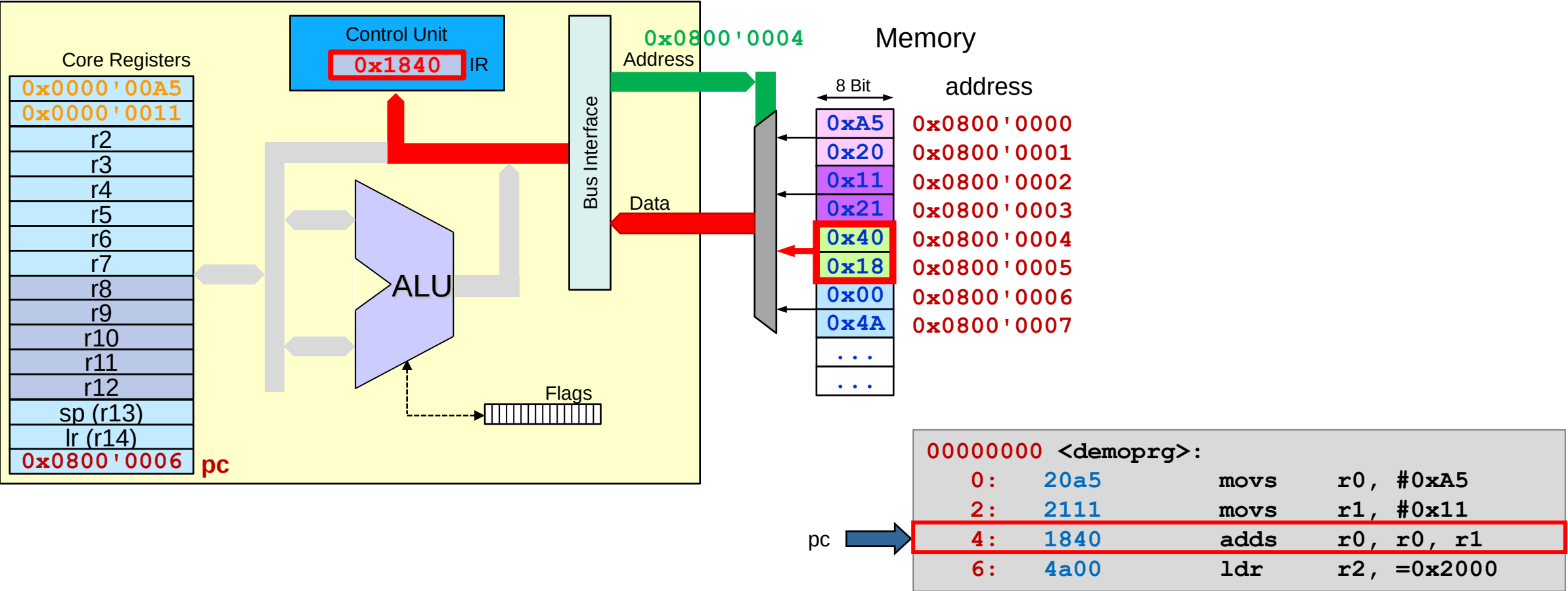
Program Execution

– pc → Address Bus



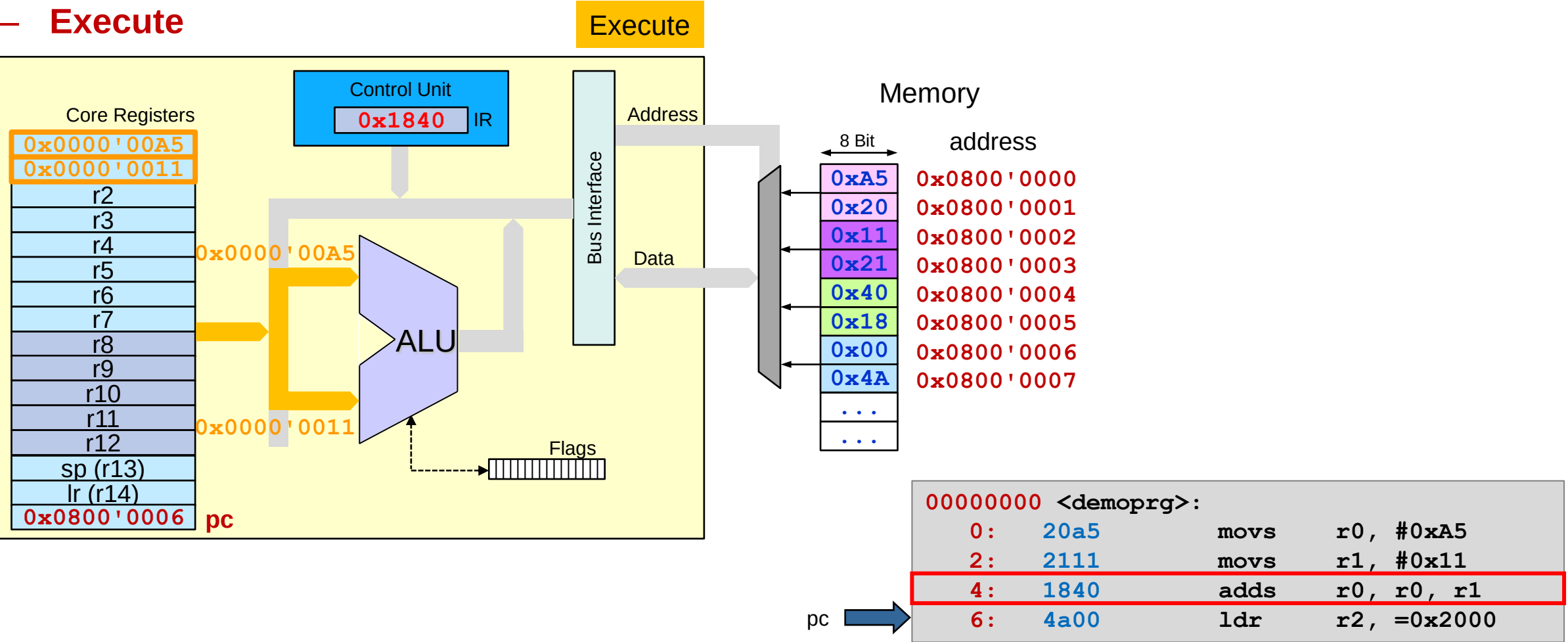
Program Execution

– Read Instruction / Increment pc Fetch



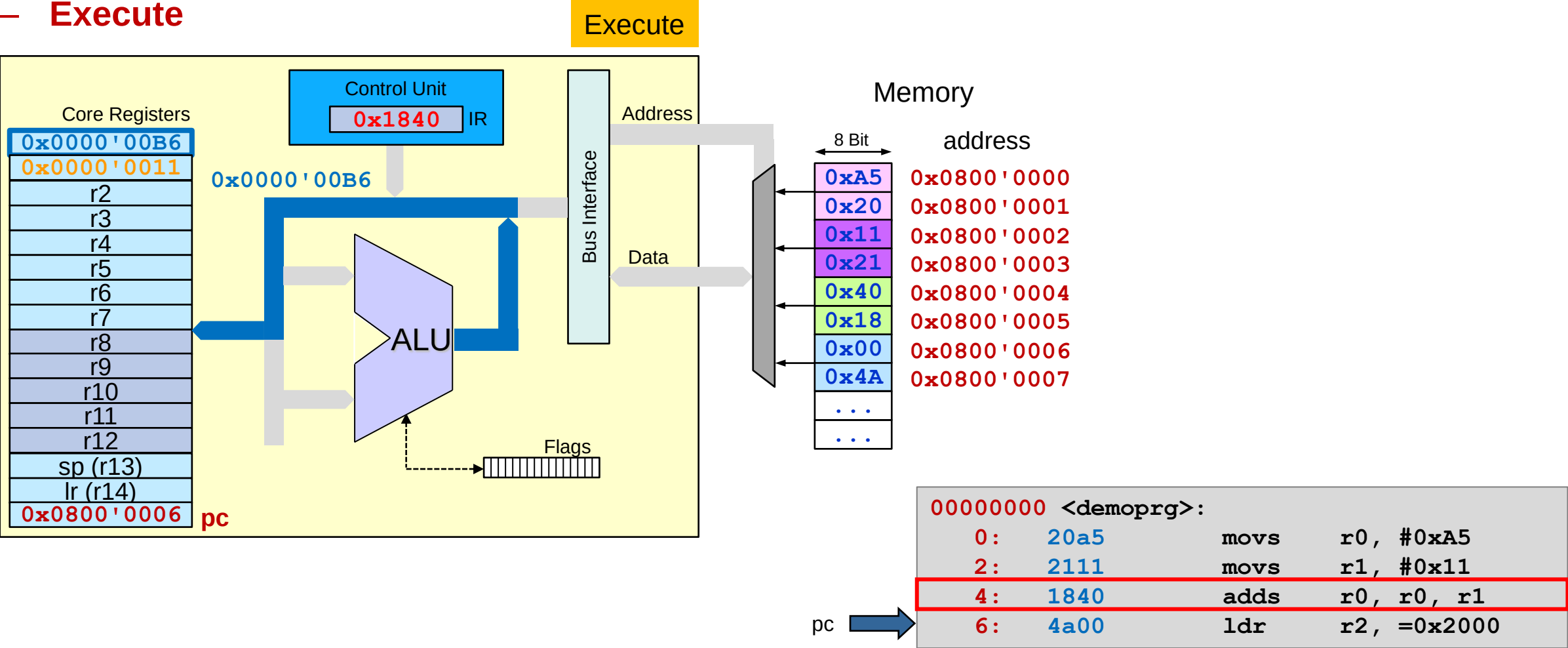
Program Execution

– Execute



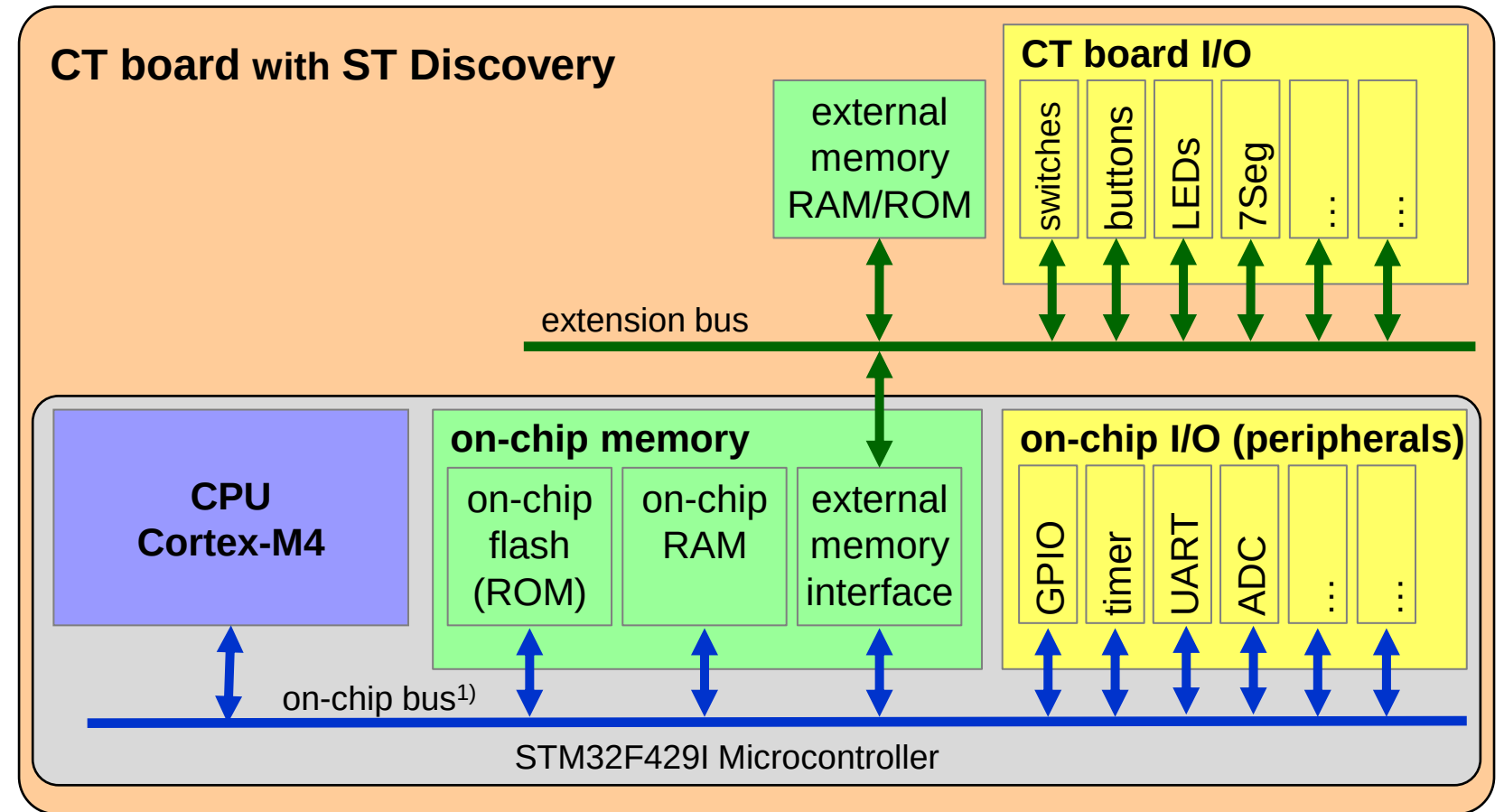
Program Execution

– Execute



Memory Map

Hardware View



¹⁾ The implementation partitions the on-chip bus into several busses called AHBx and APBx

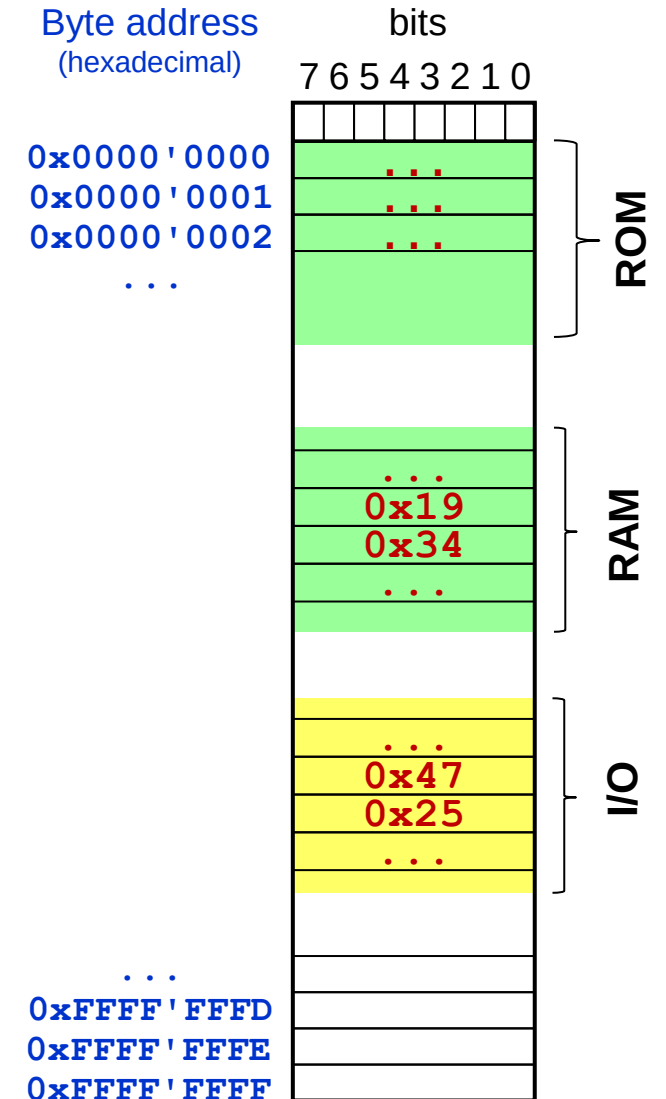
Memory Map

Memory Layout

- System Address Map
- Graphical layout of main memory
- Linear array of bytes
- What is located where (at which address) in memory?
 - Location of RAM
(readable and writable)
 - Location of ROM
(only readable)
 - Location of I/O registers

Memory maps in CT1/CT2 will be drawn with lowest address at the top and highest address at the bottom. This simplifies work with assembly listing and tables.

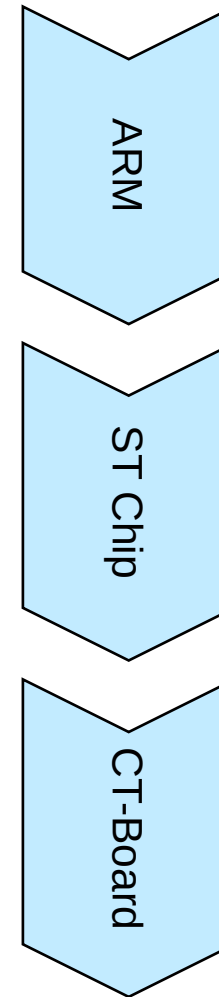
Caution: ARM and ST documentation are the other way round. Lowest address at the bottom and highest address at top.



Memory Map

Address Allocation

- ARM policies
 - Cortex-M specific
 - guidelines for chip manufacturer
- ST design decisions
 - chip specific
 - number and size of on-chip RAMs
 - size of flash
 - control register for peripherals
- CT board design decisions
 - board specific
 - LEDs, switches, etc.



Memory Map CT-Board

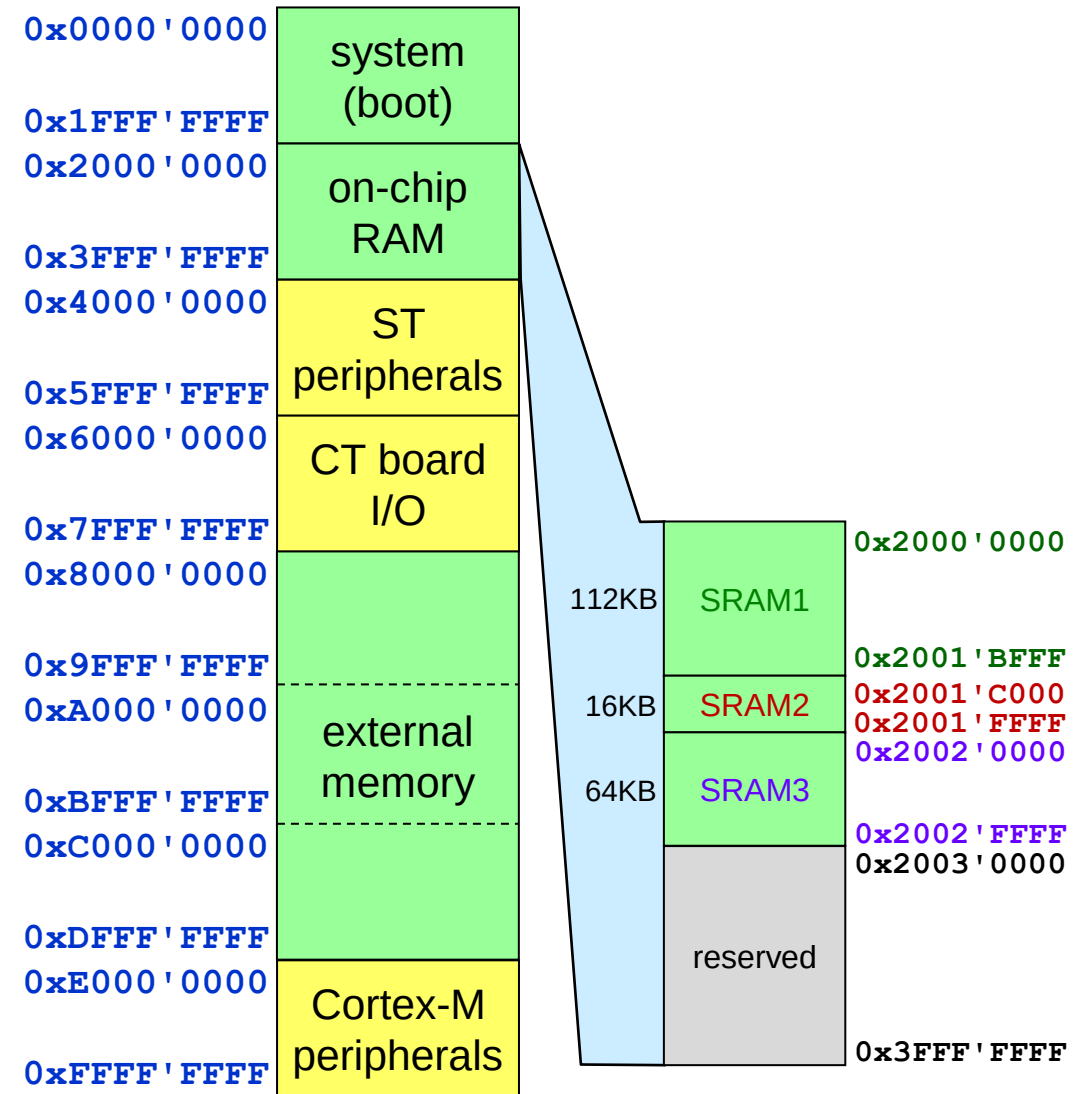
- Address space: 4 GByte = 2^{32} Byte
- From 0x0000'0000 to 0xFFFF'FFFF
- Partitioned into 8 blocks of 512 MByte each

0x0000'0000	system (boot)
0x1FFF'FFFF 0x2000'0000	on-chip RAM
0x3FFF'FFFF 0x4000'0000	ST peripherals
0x5FFF'FFFF 0x6000'0000	CT board I/O
0x7FFF'FFFF 0x8000'0000	
0x9FFF'FFFF 0xA000'0000	external memory
0xBFFF'FFFF 0xC000'0000	
0xDFFF'FFFF 0xE000'0000	
0xFFFF'FFFF	Cortex-M peripherals

Memory Map

ST chip specific

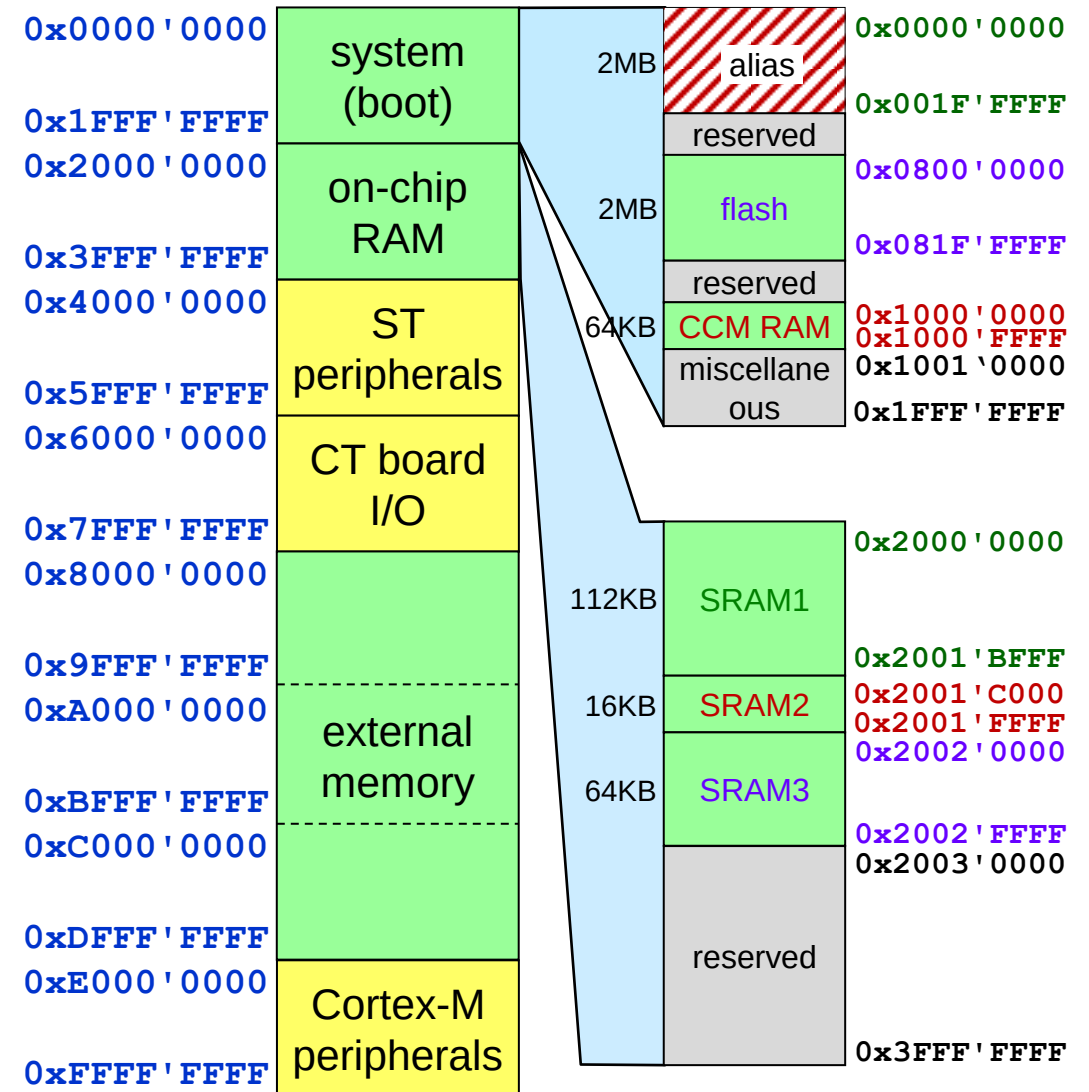
- SRAM1
 - 112 KByte
- SRAM2
 - 16 KByte
- SRAM3
 - 64 KByte



Memory Map

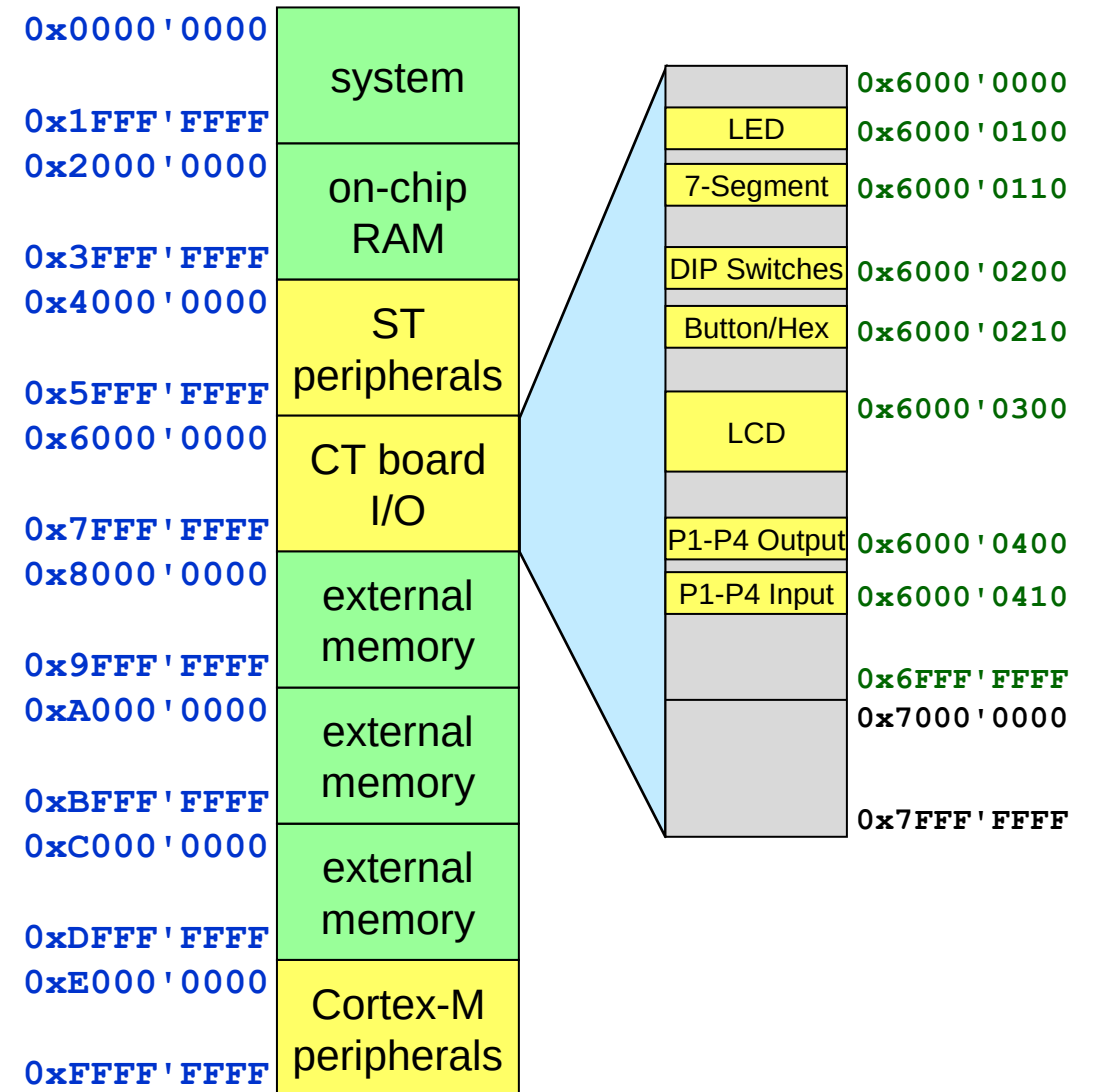
ST chip specific

- Flash
 - non-volatile memory
- CCM RAM
 - core coupled memory
 - very fast RAM
- Alias
 - user configurable mirror
 - physical memory can appear at two locations



Memory Map

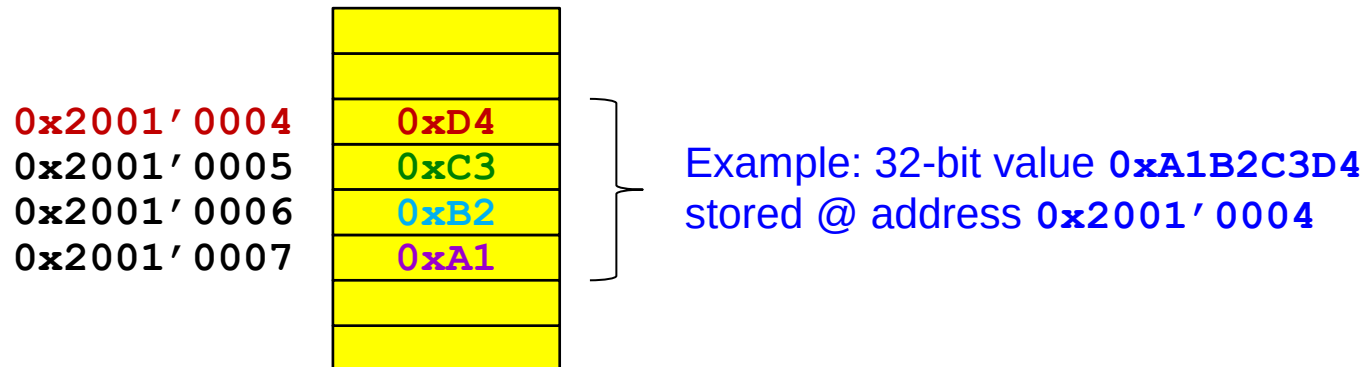
CT-Board I/O



Integer Types

Multi-byte Integer Types

- Usually memory is organized in bytes
 - one address per byte → reasons: space and history
- An integer type often requires several bytes
 - e.g., 4 byte addresses are required to store a 32-bit integer



Integer Types

Integer Types in C

- Sizes of integer types are platform dependent

Sizes in byte

8051

char	1
short	2
int	2
long int	4
char *	2

Cortex-Mx: Keil (ARM)

char	1
short	2
int	4
long int	4
long long int	8
void *	4

x86-64 (i7): gcc

char	1
short	2
int	4
long int	8
long long int	8
void *	8

Integer Types

ARM Cortex-M

C99 / specified width



C-Type – unsigned integers	Size	Term	inttypes.h / stdint.h
<code>unsigned char</code>	8 Bit	Byte	<code>uint8_t</code>
<code>unsigned short</code>	16 Bit	Half-word	<code>uint16_t</code>
<code>unsigned int</code>	32 Bit	Word	<code>uint32_t</code>
<code>unsigned long</code>	32 Bit	Word	<code>uint32_t</code>
<code>unsigned long long</code>	64 Bit	Double-word	<code>uint64_t</code>

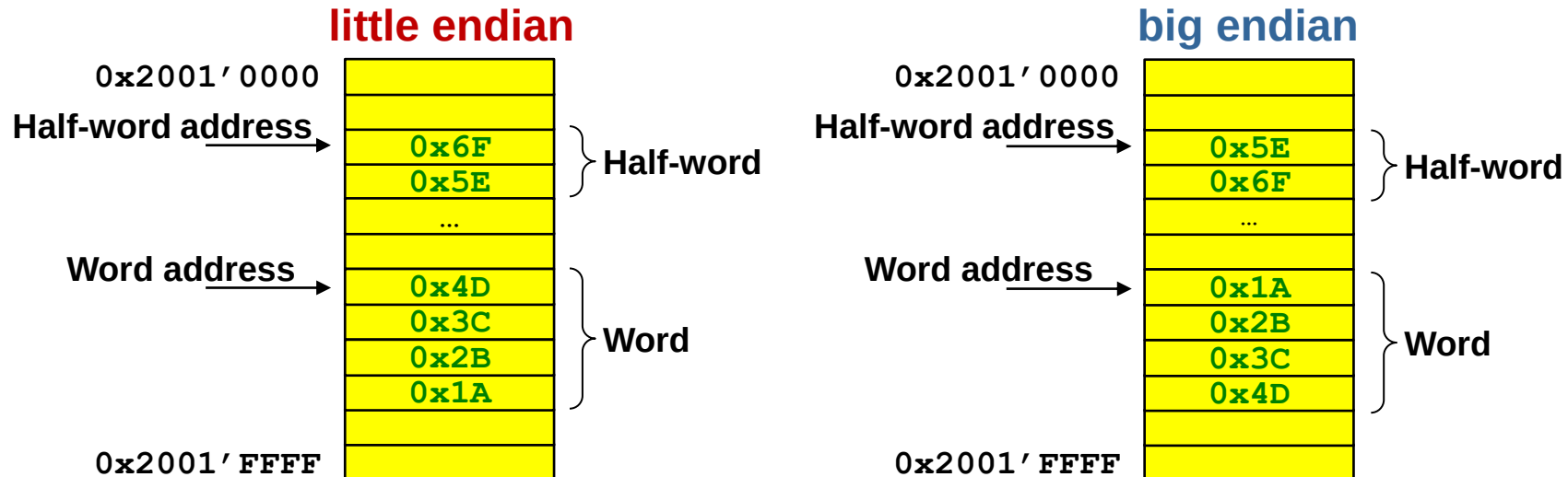
C-Type – signed integers	Size	Term	inttypes.h / stdint.h
<code>signed char</code>	8 Bit	Byte	<code>int8_t</code>
<code>short</code>	16 Bit	Half-word	<code>int16_t</code>
<code>int</code>	32 Bit	Word	<code>int32_t</code>
<code>long</code>	32 Bit	Word	<code>int32_t</code>
<code>long long</code>	64 Bit	Double-word	<code>int64_t</code>

Integer Types

How are groups of bytes arranged in memory?

- little endian → least significant byte at lower address
e.g. Intel x86, Altera Nios, ST ARM (STM32)
- big endian → most significant byte at lower address
e.g. Freescale (Motorola), Powerpc

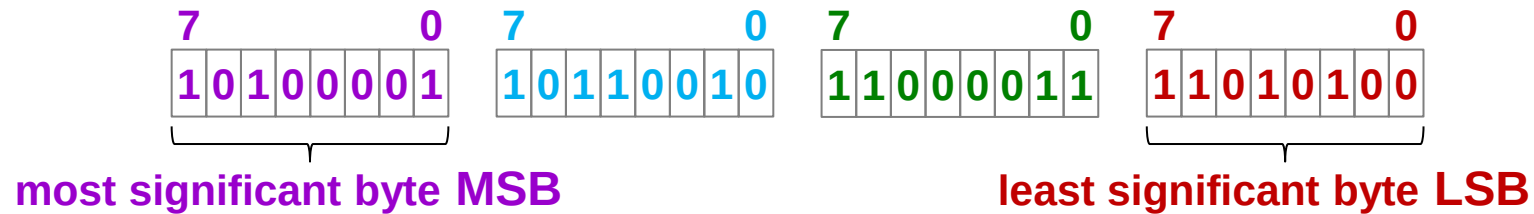
Examples: **0x1A2B'3C4D** for Word and **0x5E6F** for Half-word



Integer Types

Example

- Store Word $0xA1B2' C3D4$ at Address $0x2001' 0004$



little endian

$0x2001' 0004$	$0xD4$
$0x2001' 0005$	$0xC3$
$0x2001' 0006$	$0xB2$
$0x2001' 0007$	$0xA1$

big endian

$0x2001' 0004$	$0xA1$
$0x2001' 0005$	$0xB2$
$0x2001' 0006$	$0xC3$
$0x2001' 0007$	$0xD4$

Integer Types

Alignment

- Half-word aligned Variables aligned on even addresses
- Word aligned Variables aligned on addresses that are divisible by four

Word Aligned

0x2001'0000

0x2001'0004

0x2001'0008

Half-word Aligned

0x2001'0000

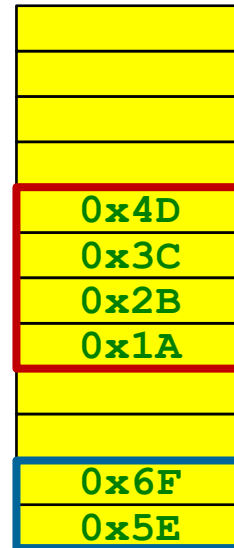
0x2001'0002

0x2001'0004

0x2001'0006

0x2001'0008

0x2001'000A

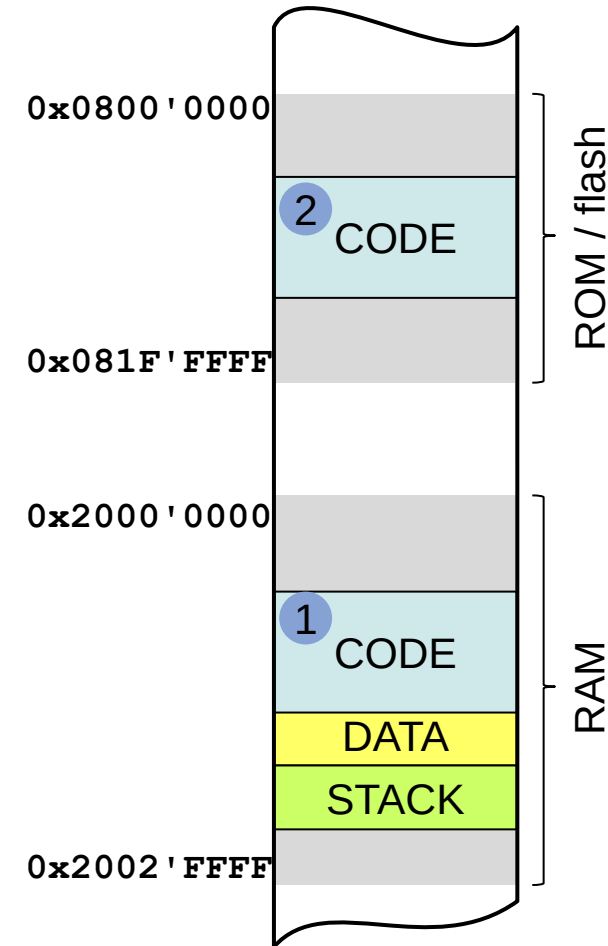


Object File Sections

- **CODE** ① ②
 - Read-only → RAM or ROM
 - Instructions (opcodes)
 - Literals ¹⁾
- **DATA** ²⁾
 - Read-write → RAM
 - Global variables
 - *static* variables in C
 - Heap in C → `malloc()`
- **STACK**
 - Read-write → RAM
 - Function calls / parameter passing
 - Local variables and local constants

¹⁾ Literal: a fixed/constant value in source code

²⁾ The data section is sometimes further sub-divided into a read-only and a read-write part



Object File Sections

Assembly Program Structure

Define code area to include your program

```
.text
.global _start
_start:
    movs    r4, #12
    b       _start
```

Define data area to store global variables, etc.

```
.data
byte_var:  .byte    0x1A, 0x00
hw_var:    .hword   0x2B3C
word_var:  .word    0x4D5E6F70
```

Define data area (readonly)

```
.rodata
b_const:   .byte    0x1A, 0x00
hw_const:  .hword   0x2B3C
w_const:   .word    0x4D5E6F70
```

Define stack area to reserve space for stack

```
.bss
.align    2
stack_mem:
.skip     0x400
```

Object File Sections

.section directive

Information only

- Define a section with name, flags and attributes
 - flags and type are optional

```
.section <name>, "<flags>", %<type>
```

"<flags>"	meaning
"a"	allocate memory
"w"	write
"x"	execute
"M"	Merge
"S"	strings

- Examples:

section	.section directive (typical)	Syntax sugar
code	<code>.section .text, "ax", %progbits</code>	<code>.text</code>
data	<code>.section .data, "aw", %progbits</code>	<code>.data</code>
uninitialized	<code>.section .bss, "aw", %nobits</code>	<code>.bss</code>

%<type>	meaning
%progbits	normal data/code
%nobits	uninitialized values

Object File Sections

Memory Map

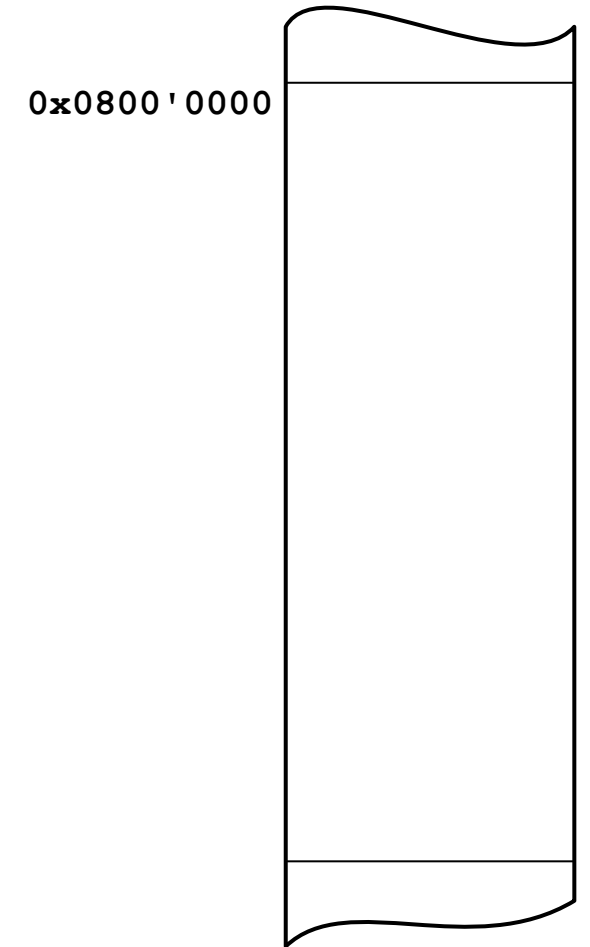
Assume

- A program uses the following memory segments during execution

- Code 0x0800'1000 to 0x0800'17FF
- Data 0x2001'0000 to 0x2001'01FF
- Stack 0x2001'0200 to 0x2001'05FF

– Exercise

- Draw the memory map with the three sections
- For each section mark the first and the last address with its value
- How many memory cells (bytes) does each section contain?
- For each section: In which STM32F4 memory is it located?

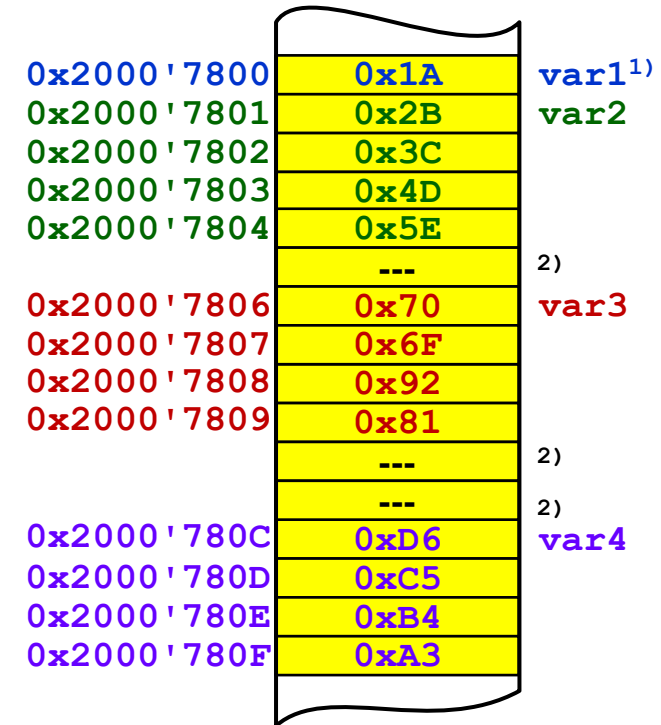


Object File Sections

Memory Allocation in Assembly

- Memory Allocation in Assembly
 - Directives for initialized data
 - `.byte` bytes
 - `.hword` half-words (half-word aligned)
 - `.word` words (word aligned)
 - can be located in `.data`, `.rodata`, or `.text`

```
.data
var1:
    .byte 0x1A
var2:
    .byte 0x2B, 0x3C, 0x4D, 0x5E
var3:
    .hword 0x6F70, 0x8192
var4:
    .word 0xA3B4C5D6
```



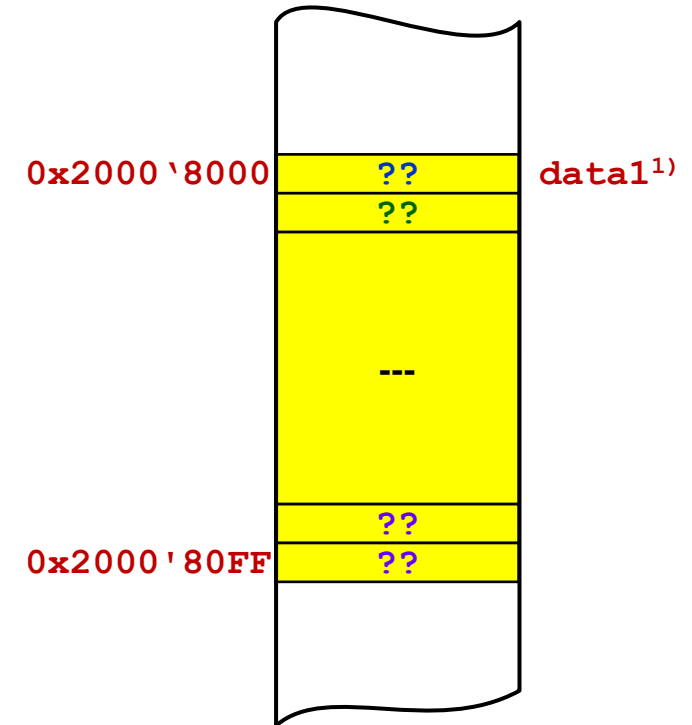
¹⁾ if we assume that example1 starts at 0x2000'7800

²⁾ Padding bytes introduced for alignment

Object File Sections

- **Memory Allocation in Assembly**
 - Directives for uninitialized data
 - **skip** or **%** with number of bytes to be reserved
 - Reserves number of bytes without initializing them

```
.data
data1:
    .skip 256
```



¹⁾ if we assume that example2 starts at 0x2000'8000

Object File Sections

Code Example

```
show_variables.c
uint32_t  g_init_var = 0x4D5E6F70;
uint32_t  g_noinit_var;
const uint32_t g_const = 0xDDEEFF00;
void show_variables(void)
{
    uint32_t local_var;

    const uint32_t local_const =
                                0x7261504F;

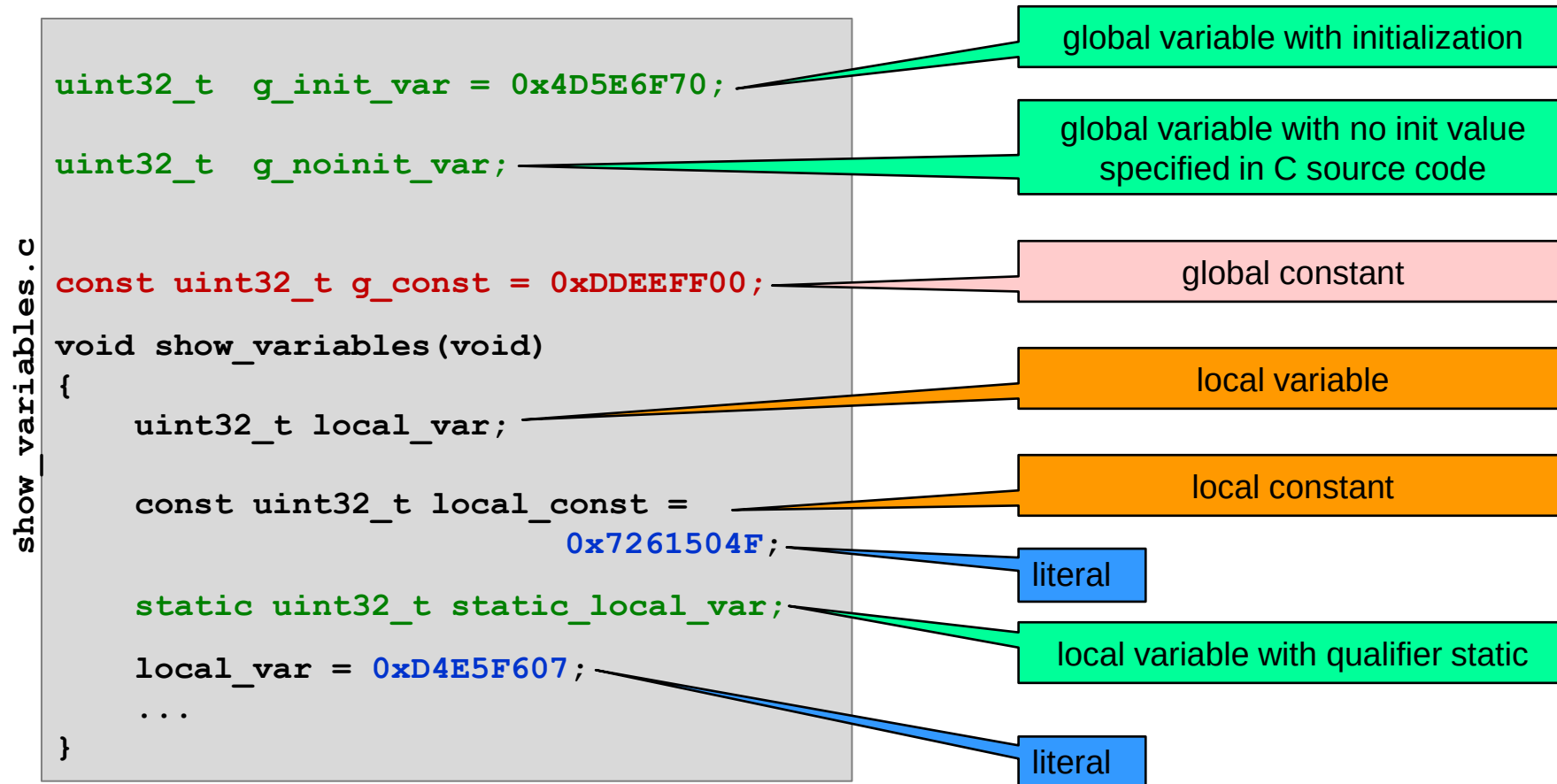
    static uint32_t static_local_var;

    local_var = 0xD4E5F607;
    ...
}
```

Into which sections will the colored objects be allocated?

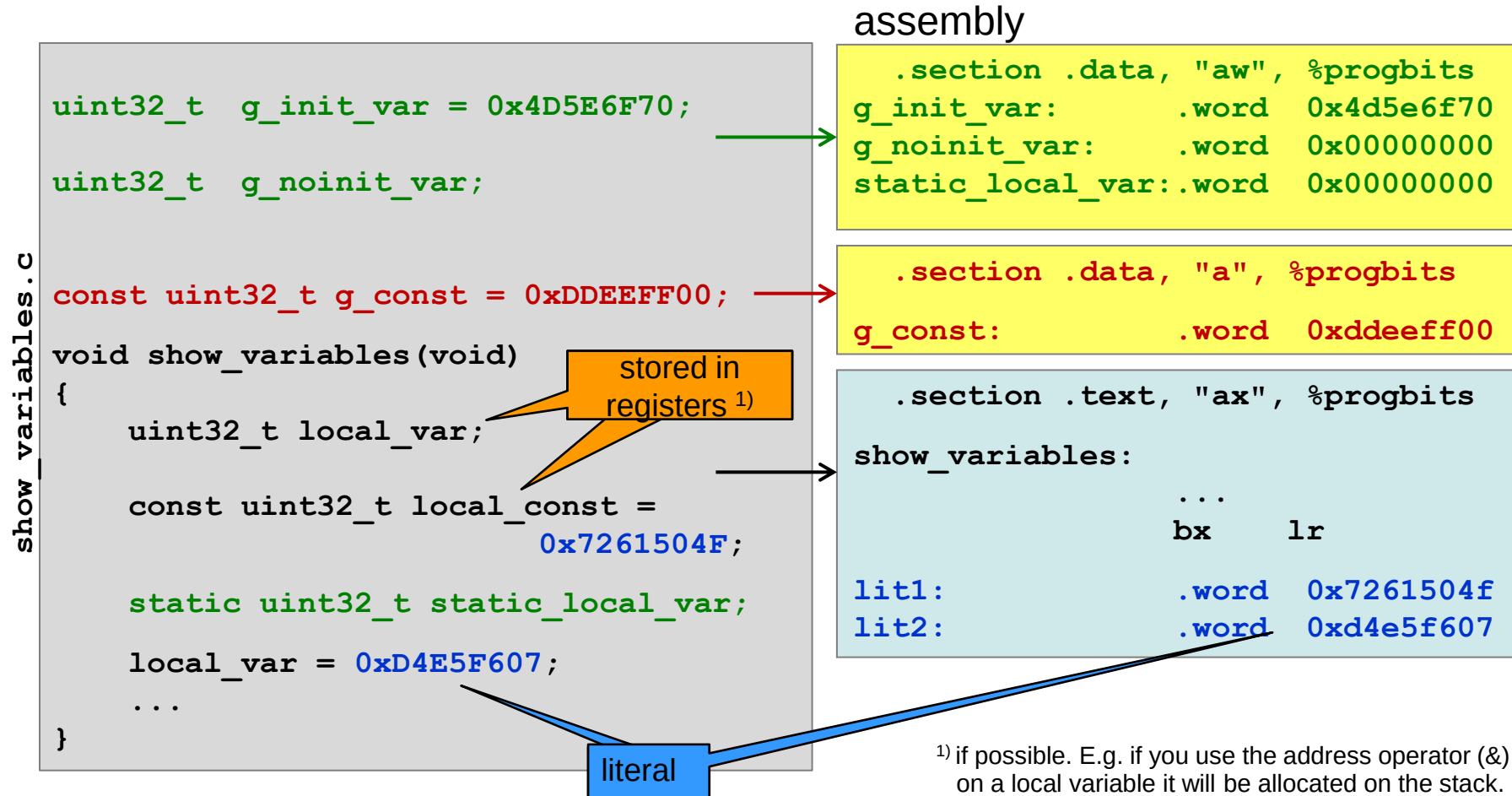
Object File Sections

Code Example



Object File Sections

Code Example



¹⁾ if possible. E.g. if you use the address operator (&) on a local variable it will be allocated on the stack.

Conclusion

- **Components Cortex-M CPU**
 - Core Registers: r0-r12, sp, lr, pc
 - 32-bit ALU
 - Flags (APSR)
- **Instruction Types**
 - **Data transfer, data processing, control flow**
- **Program Execution**
 - Fetch – Execute
- **Memory Map**
- **Integer Types**
 - Size depends on architecture → use C99 types for portability
 - 'Little Endian' vs. 'Big Endian', alignment
- **Object File Sections**
 - .text, .data, .bss
- Control Unit with IR (Instruction Register)
- Bus Interface